Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Воронежский государственный технический университет»
(ФГБОУ ВО «ВГТУ», ВГТУ)

Факультет информационных технологий и компьютерной безопасности
Кафедра систем информационной безопасности
Направление подготовки (специальность) 10.05.03 Информационная безопасность автоматизированных систем
(код и наименование направления подготовки)
Профиль/программа/направленность Обеспечение информационной безопасности распределённых информационных систем

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Дипломная работа
(вид выпускной квалификационной работы – бакалаврская работа, дипломная работа (проект), магистерская диссертация)
Швецов Владимир Алексеевич
(фамилия, имя, отчество обучающегося)

Тема: Управление рисками успешности реализации кибератак: алгоритмическое и программное обеспечение

Состав выпускной квалификационной работы:

Расчетно-пояснительная записка на 71 страницах
Графическая часть на _____ листах

Расчетно-пояснительная записка к выпускной квалификационной работе:

Заведующий кафедрой _____ (А.Г. Остапенко)
(подпись) (инициалы, фамилия)

Руководитель _____ (А.С. Пахомова)
(подпись) (инициалы, фамилия)

Консультанты:
по _____ (_____)
(наименование раздела, подпись) (инициалы, фамилия)

по _____ (_____)
(наименование раздела, подпись) (инициалы, фамилия)

по _____ (_____)
(наименование раздела, подпись) (инициалы, фамилия)

по _____ (_____)
(наименование раздела, подпись) (инициалы, фамилия)

по _____ (_____)
(наименование раздела, подпись) (инициалы, фамилия)

Обучающийся _____ В.А. Швецов
(подпись) (инициалы, фамилия)

«02» 07 2025 г.

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВГТУ», ВГТУ)

Факультет информационных технологий и компьютерной безопасности
Кафедра систем информационной безопасности
Направление подготовки/специальность 10.05.03 Информационная безопасность автоматизированных систем
(код, наименование)
Профиль/программа/направленность Обеспечение информационной безопасности распределённых информационных систем

Утверждаю _____
(дата)
Зав.кафедрой _____
(подпись)

ЗАДАНИЕ
ПО ПОДГОТОВКЕ ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ

Дипломная работа
дипломная работа (проект), бакалаврская работа, магистерская диссертация

Обучающемуся Швецову Владимиру Алексеевичу
фамилия, имя, отчество

1. Тема: Управление рисками успешности реализации кибератак: алгоритмическое и программное обеспечение

ФГБОУ ВО «ВГТУ», ВГТУ
по заданию какой организации выполняется работа

приказ №01-1-10/3-2979 от 25.06.2025
утверждена приказом по университету

2. Срок сдачи обучающимся выпускной квалификационной работы

3. Исходные данные

4. Краткое содержание выпускной квалификационной работы

Раздел 1 Алгоритмы оценки рисков кибератак
перечень подлежащих разработке в выпускной квалификационной работе вопросов
Раздел 2 Алгоритмы управления успешностью реализации кибератак
Раздел 3 Проектирование программного обеспечения управления рисками

5. Перечень графического материала (с точным указанием обязательных чертежей по разделам

6. Консультанты (с указанием относящихся к ним разделов)

График выполнения выпускной квалификационной работы

№ п/п	Наименование разделов выпускной квалификационной работы	Срок выполнения этапов работы
1	Введение	12.03-22.05
2	Алгоритмы оценки рисков кибератак	22.03-22.05
3	Алгоритмы управления успешностью реализации кибератак	23.05-31.05
4	Проектирование программного обеспечения управления рисками	01.06-24.06
5	Заключение	25.06-02.07

Руководитель _____
(подпись)

Дата выдачи задания 22.03.25

Задание принял к исполнению 22.03.25
(дата)

Подпись обучающегося _____

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Воронежский государственный технический университет»

АННОТАЦИЯ
выпускной квалификационной работы

Обучающегося Швецова Владимира Алексеевича

Тема ВКР: Управление рисками успешности реализации кибератак: алгоритмическое и программное обеспечение

Характеристика выпускной квалификационной работы

Актуальность исследования: острая нехватка квалифицированных специалистов в области кибербезопасности, требует кардинальных решений для стабилизации информационного поля, однако повлиять на это очень трудно. Разработка и создание алгоритмического и программного обеспечения для управления рисками может значительно улучшить производительность труда существующих специалистов и уменьшить масштаб проблемы в виде дефицита кадров в области информационной безопасности.

Задачи, решаемые в ВКР: анализ существующих аналогов приложений и алгоритмов в области управления рисками; разработка принципиально нового алгоритма калькуляции и управления рисками успешности реализации кибератак; программная реализация разработанного алгоритма в виде понятного и простого в использовании программного обеспечения с возможностью визуализации.

Структура ВКР: исследование существующих аналогов алгоритмов управления рисками; разработка алгоритмов управления рисками успешности реализации кибератак; проектирование программного обеспечения управления рисками.

Краткая характеристика полученных результатов: рассмотрены существующие алгоритмы управления рисками, выявлены их недостатки и ограничения, сформированы требования к новому алгоритму; разработан новый алгоритм впервые с использованием гипергеометрического распределения для калькуляции и регулирования риска; разработано и представлено программное обеспечение, в полной мере реализующее функции алгоритма.

Автор ВКР_____ Швецов Владимир Алексеевич
(подпись)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 7
1 Алгоритмы оценки рисков кибератак 12
1.1 Существующие алгоритмы управления рисками 12
1.1.1 Модель FAIR (Factor Analysis of Information Risk) 12
1.1.2 Платформа SOAR (Security Orchestration, Automation, and Response) 14
1.2 Проблемы существующих алгоритмов 15
1.2 Потребность разработки нового алгоритма 17
1.3 Требования к разрабатываемому алгоритму управления рисками успешности реализации кибератак 18
2 Алгоритмы управления успешностью реализации кибератак 19
2.1 Алгоритм управления рисками с использованием биноминального распределения 19
2.1.1 Биноминальное распределение 19
2.1.2 Разработка алгоритма управления рисками 19
2.1.3 Последовательность выполнения действий 35
2.1.4 Область применения алгоритма 39
2.2 Алгоритм управления рисками с использованием гипергеометрического распределения 39
2.2.1 Гипергеометрическое распределение 39
2.2.2 Разработка алгоритма управления рисками 41
2.2.3 Последовательность выполнения действий 49
2.2.4 Область применения алгоритма 53
3 Проектирование программного обеспечения управления рисками 54

3.1 Общее описание архитектуры ПО 54
3.2 Технологическое обеспечение 55
ЗАКЛЮЧЕНИЕ 56
СПИСОК ЛИТЕРАТУРЫ 57
ПРИЛОЖЕНИЕ А 64

ВВЕДЕНИЕ

В настоящее время сфера информационной безопасности динамично развивается и постоянно сталкивается с новыми проблемами. Главные из них это рост количества киберугроз и нехватка квалифицированных специалистов [1-9].

Стремительная цифровизация самых разных отраслей способствует стремительному увеличению количества кибератак. Компьютерные сети собирают всё больше информации, которой желают завладеть злоумышленники. Остановить или хотя бы замедлить рост преступной деятельности в информационном поле могут лишь высококвалифицированные специалисты в области информационной безопасности (ИБ). Спрос на них в последнее время действительно сильно возрос, но вот предложение категорически не поспевает [15].

Одна из ведущих организаций в области ИБ, занимающаяся сертификацией специалистов в этой области, - ISC2 ежегодно опубликует отчеты о состоянии рынка труда в кибербезопасности. Из последнего отчета, в котором опубликованы данные за прошедший 2024 год, можно узнать, что количество специалистов в области ИБ выросло всего лишь на 0,1%, и это при том, что дефицит на таких специалистов за тот же период получил прирост в виде 19,1%. Также указывается, что на момент публикации отчета общемировая нехватка квалифицированных специалистов составляла 4,7 миллиона сотрудников. И это при 5,4 миллионах уже находящихся и работающих в сфере ИБ. Основываясь на данных в опросе, который ISC2 провела среди почти 16 тысяч профессионалов по кибербезопасности, большинство из них готовы взять на работу сотрудника, который имеет хоть какой-то опыт в этой сфере, и готовы всеми силами помочь достичь ему необходимого уровня понимания и знаний, чтобы стать ценным кадром для организации [16]. Данные условия на рынке труда в области ИБ наглядно демонстрируют остроту проблемы нехватки кадров.

Очевидно, что проблема слишком масштабна, чтобы для неё существовало какое-либо быстрое и простое решение: заниматься этим вопросом придется долго и подход к нему нужен комплексный.

Однако, одним из потенциальных решений можно рассмотреть оптимизацию труда имеющихся работников. Так, можно создавать более комфортные для них условия путем автоматизации рутинных процессов.

Первым шагом на пути к автоматизированному выполнению определенных процессов станет их алгоритмизация. Замена теоретических знаний и бесчисленных методик на точные и предсказуемые алгоритмы значительно повышает эффективность труда, ведь грамотно разработанный алгоритм гарантирует:

– четкую последовательность действий для решения конкретной, строго определенной задачи;

– детерминированность (предопределенность), которая означает, что при одних и тех же входных данных алгоритм всегда будет выдавать один и тот же результат;

– достижение цели, получения конкретного предсказуемого результата;

– скорость выполнения – строгая регламентированность точно выполняемых, понятных шагов

– корректную работу программного обеспечения (ПО) на его основе, которое, в свою очередь, не только ещё сильнее ускорит процесс обработки данных, но и исключит ошибки человеческого фактора в этом процессе.

Таким образом, актуальность темы исследования главным образом обусловлена острой нехваткой квалифицированных специалистов в области кибербезопасности, повлиять на которую очень трудно, однако можно значительно улучшить производительность труда существующих специалистов за счёт разработки и создания алгоритмического и программного обеспечения управления рисками. Также актуальность подкреплена наличием следующих противоречий между:

– динамикой роста количества угроз информационной безопасности и частотой обновления и выпуска новых решений по её обеспечению;

– необходимостью проведения точных и безошибочных вычислений и высокой сложностью в совокупности с низкой скоростью их выполнения ручными расчетами;

– многофакторностью природы риска и отсутствием понятных алгоритмов для качественного управления им в условиях, максимально приближенных к реальным.

Работа посвящена разработке алгоритмов и их аппаратной реализации в виде программного обеспечения для управления рисками успешности реализации кибератак. В работе рассматриваются:

– существующие аналоги алгоритмов подобного назначения;

– использование двух различных математических распределений для калькуляции значений риска с последующей адаптацией вычислений с целью управления конечным результатом;

– разработка и создание удобного, эффективного и понятного пользователю программного обеспечения для управления рисками.

Основной акцент выполнен на создании алгоритма, наиболее приближенного к реальным условиям функционирования информационных систем и наиболее применимого в них.

Объектом исследования являются информационные сети и системы, подверженные различным видам кибератак.

Предмет исследования – алгоритмическое и программное обеспечения управления рисками успешности реализации кибератак на информационные объекты.

Основной целью исследования является повышение защищенности информационных сетей и систем за счет разработки алгоритмического и программного обеспечения управления рисками успешности реализации кибератак.

Для достижения поставленной цели необходимо выполнить следующий ряд задач:

– анализ существующих аналогов приложений и алгоритмов в области управления рисками;

– разработка принципиально нового алгоритма калькуляции и управления рисками успешности реализации кибератак;

– программная реализация разработанного алгоритма в виде понятного и простого в использовании программного обеспечения с возможностью визуализации.

В соответствии с вышепоставленными задачами исследования на защиту выносятся следующие результаты:

– недостатки и ограничения существующих алгоритмов, а также обоснование для разработки нового алгоритма управления рисками успешности реализации кибератак;

– разработаны два полноценных алгоритма вычисления и управления рисками, основанные на различных математических распределениях;

– наиболее подходящий под реальные условия алгоритм получил реализацию в виде программного обеспечения.

Новизна исследования:

– проведен анализ существующих алгоритмов по управлению рисками информационной безопасности, определены их слабые стороны и недостатки, сформированы требования к алгоритму, который будет оставаться актуальным намного дольше;

– впервые для управления рисками успешности реализации кибератак представлен алгоритм, основанный на гипергеометрическом распределении;

– алгоритм нового образца получил автоматизацию.

Теоретическая значимость результатов исследования заключается в системном подходе к анализу факторов риска для разработки нового алгоритма управления им. Данная работа вносит вклад в развитие теоретических знаний в следующих аспектах:

– исследование выявило не только слабые стороны современных аналогов разработанному алгоритму, но и их причины, что позволит исключить эти недостатки ещё на этапе разработки новых методик и алгоритмов;

– разработаны два совершенно новых алгоритма управления рисками с использованием математических распределений и современных подходов к рассмотрению факторов риска, что открывает новый взгляд на них;

– предложен вариант программного обеспечения, способный в автоматическом режиме просчитывать и регулировать риск по заданным параметрам.

Практическая ценность работы состоит в том, что её результаты можно использовать в качестве:

– учебных материалов по информационной безопасности на кафедрах различных вузов;

– отправной точки для проектирования более масштабного программно-аппаратного комплекса по управлению рисками успешности реализации кибератак;

– программного обеспечения для расчета риска и его управления сотрудниками отделов ИБ в различных организациях.

1 Алгоритмы оценки рисков кибератак

1.1 Существующие алгоритмы управления рисками

1.1.1 Модель FAIR (Factor Analysis of Information Risk)

FAIR (Factor Analysis of Information Risk) – это количественная модель оценки рисков, которая разработана для анализа угроз информационной безопасности в финансовых терминах. В отличие от традиционных качественных методов работы и информационными рисками, FAIR предлагает структурированный метрический подход, позволяющий выражать риски через вероятностные распределения и денежные величины [10].

Данная модель была создана Джеком Джонсоном в 2005 году и с того времени получила широкое распространение в корпоративном риск-менеджменте, особенно в сферах, требующих обоснования инвестиций в кибербезопасность. Модель является актуальной и по сей день [11].

Модель FAIR декомпозирует риск на четыре ключевых фактора:

– частота угроз (Threat Event Frequency, TEF) – определяет, как часто злоумышленник может инициировать атаку. Значение данного компонента зависит от мотивации злоумышленника, доступности уязвимостей и эффективности текущих защитных мер информационной системы;

– вероятность успеха атаки (Vulnerability, VULN) – значение, которое оценивает шансы на успех при реализации атаки. Оно рассчитывается на основании наличия в системе защитных механизмов, уровня подготовки сотрудников, сложности эксплуатации выбранной злоумышленником уязвимости;

– величина потерь (Loss Magnitude, LM) – компонента, определяющая финансовые последствия успешной атаки, также включает в себя первичные (штрафы, восстановления инфраструктуры) и вторичные потери (репутационный ущерб, судебные издержки);

– факторы воздействия (Threat Capability, Resistance Strength), которые учитывают уровень профессионализма атакующих и способность организации оказывать сопротивление атаке.

Математически модель FAIR можно представить следующей формулой (1):

$$Risk = \sum (TEF \cdot VULN \cdot LM), \quad (1)$$

где TEF – частота угроз, выражается в количестве событий в год;

VULN – вероятность успеха атаки, имеет значение от 0 до 1;

LM – финансовый ущерб, который выражается в валютном эквиваленте.

Таким образом, например, если в организации существует угроза утечки данных по причине реализации злоумышленником фишинговой атаки, её частота в среднем составляет 10 раз в год, успешность реализации 30%, а финансовые затраты оцениваются в 200 000 рублей, то риск, согласно формуле 1, составит 600 000 рублей [11].

Данная модель очень популярна и эффективна в финансовой деятельности, она позволяет обосновывать затраты на безопасность, применима к любым типам угроз, будь то DDoS-атаки, инсайдеры или APT, однако в ней отсутствует как таковое управление риском.

Также модель FAIR имеет серьезные ограничения, снижающие ее эффективность в реальных условиях. Во-первых, проблема недостатка данных, которая обусловлена тем, что модель требует точных статистических данных по атакам, сбор которых затруднен, а в случае возникновения новых угроз оценки и вовсе становятся субъективными. Во-вторых, модель сложна к внедрению в организацию, так как требует привлечения экспертов по анализу рисков и трудоемкого ручного вычисления в отсутствие специализированного ПО.

Таким образом, FAIR это эффективный инструмент для оценки рисков ИБ, но не для управления ими.

1.1.2 Платформа SOAR (Security Orchestration, Automation, and Response)

SOAR (Security Orchestration, Automation, and Response) – класс платформ, объединяющих различные инструменты, автоматизацию процессов и реагирование на инциденты. В отличие от моделей оценки рисков (например FAIR), SOAR реализует алгоритмы управления через динамическое принятие решений.

SOAR платформа функционирует как цикл обработки инцидентов, включающий в себя следующие шаги:

- сбор данных;
- корреляция событий;
- оценка риска;
- автоматизированное реагирование;
- пост-анализ.

В свою очередь реализация такого цикла функционирования требует особой структуры платформы. SOAR содержит в себе такие алгоритмические модули, как:

- объединяющий механизм, который поддверживает интеграцию с более чам 50 источниками (SIEM, EDR, Firewall, Threat Intel);
- ядро принятия решений, состоящее из линейных сценариев (Playbooks) и адаптивных решений (ML-модели);
- исполнитель действий, который может в автоматическом режиме выполнять блокировку IP-адресов, изоляцию узлов и сброс сессий.

На основании комплексного анализа собранных данных SOAR оценивает состояние инфраструктуры организации, выявляет подозрительные события, распределяет их по степени риска и информирует о них специалистов по ИБ. Если будет необходимо, система может изолировать зараженные устройства, чтобы не допустить дальнейшего распространения атаки.

С помощью автоматических сценариев или в ручном режиме SOAR-платформа дополняет информацию об инциденте данными из открытых внешних баз, найденных

записей об аналогичных событиях в иных системах и иных источников. На этапе анализа инцидента также формируется перечень затронутых объектов и устройств.

Реагирование на инциденты происходит на основании собранных данных, при этом производится полный набор действий, необходимых для устранения угрозы или минимизации последствий [12].

Платформы класса SOAR имеют возможность представлять собранную в ходе анализа и реагирования на инцидент информацию лицам, ответственным за информационную безопасность в организации. Также периодически формируется отчетность о текущем уровне безопасности IT-инфраструктуры.

Однако SOAR обладает рядом ограничений, которые не позволяют ему быть идеальным ИБ инструментом. Ярко выделяется проблема линейных сценариев, которые не адаптируются к изменению тактик, техник и процедур злоумышленников. В корреляции имеются слепые зоны, из-за чего не учитывается контекст бизнес-рисков, то есть критичность и ценность атакуемых активов.

Для четкой работы в обязательном порядке требуются структурированные индикаторы компрометации - наблюдаемые признаки или свидетельства, указывающие на то, что система или сеть, возможно, были скомпрометированы в результате кибератаки.

Из-за того, что SOAR-платформа это полноценный самостоятельный модуль наблюдается проблема с его интеграцией.

1.2 Проблемы существующих алгоритмов

Хоть подобных алгоритмов и существует достаточное количество, все они не лишены недостатков и ограничений.

Самая большая проблема – динамичность и адаптивность угроз. Скорость их эволюции слишком велика – злоумышленники постоянно меняют свои тактики, техники и процедуры, поэтому алгоритмы, которые протестированы на старых данных быстро теряют свою актуальность. Помимо этого, кибератаки зачастую настраиваются под конкретную жертву в обход шаблонных правил защиты, а

алгоритмы чаще всего не могут предсказать эффективную защиту от таких уязвимостей.

Другой аспект – нехватка данных об успешных атаках, чтобы понять и рассмотреть весь процесс и определить, где же был допущен просчет. Организации практически никогда не публикуют детали успешных кибератак из-за репутационных рисков и соображений безопасности, поэтому алгоритмы, в своем большинстве, оттачиваются на неудачных или обнаруженных атаках, что, непосредственно, искажает картину реальных рисков. При этом доступные данные, по понятным причинам, неполные, искаженные или нерепрезентативные для текущих угроз. Также, в некоторых ситуациях, достаточно сложно понять, действительно ли атака была остановлена, или же злоумышленник решил заморозить свой «успех», сохранив скрытое присутствие в системе для снижения внимания к нему.

Также большой пропастью в алгоритмизации является человеческий фактор. Самые сложные и продуманные алгоритмы бессильны против фишинга или манипуляции необученными пользователями, которые часто «добровольно» передают права доступа злоумышленнику. Системы, настроенные вручную их администраторами, могут иметь недочеты в политиках, что мешает алгоритмам правильно смоделировать информационные риски.

Высокий уровень ложных срабатываний и пропусков цели также является проблемой: избыточная осторожность алгоритмов приводит к потоку ложных тревог, нарушая нормальную деятельность аналитиков и снижая доверие к системе. А самый изощренные атаки, как уже было сказано, специально разработаны так, чтобы выглядеть как нормальная активность системы и избегать обнаружения. Данная проблема, известная как баланс между минимизацией ложных срабатываний и пропусков всегда была актуальная для таких алгоритмов.

Неоднородность современных информационных инфраструктур накладывает свои трудности для алгоритмов, которым трудно точно отразить все взаимосвязи и уязвимости системы. Поэтому алгоритмам сложно также предсказать, как тот или иной «успех» злоумышленника в одной системе повлияет на другие. Равнозначно

сложно смоделировать мотивацию, ресурсы, навыки и поведение реальных злоумышленников, которые способны избегать систем обнаружений и защиты.

Немаловажным является факт фундаментальной асимметрии, который заключается в том, что определенному злоумышленнику необходимо найти и использовать лишь одну единственную уязвимость или ошибку, в то время как алгоритм управления рисками обязан быть эффективным против всех возможных типов и векторов атаки.

К тому же ограничения самих технологий являются помехой в успешной реализации алгоритмов управления рисками успешности реализации кибератак. Например, все традиционные механизмы защиты, такие как антивирусы и системы обнаружения вторжений, работают с сигнатурами – шаблонами успешных атак и их попыток. Даже средства, разработанные на основе машинного обучения и искусственного интеллекта могут быть обмануты.

Фокус на технических аспектах инцидента также не позволяет современным алгоритмам спроецировать риск в реальный финансовый, репутационный или бизнес-риск.

1.2 Потребность разработки нового алгоритма

Так или иначе, не стоит думать, что современные аналоги алгоритмов управления рисками успешности реализации кибератак негодны, дело в том, что сама по себе, задача точного предсказания и управления риском во враждебной и крайне динамичной среде является экстремально сложной. Современные алгоритмы — это действительно важные инструменты, но ни один из них не может быть единственной линией обороны на пути к защищаемой информации. Их эффективность зависит от качества данных, постоянного обновления, задействования ручного труда аналитиков, использования их сильных сторон, таких как управление уязвимостями, реагирование на инциденты, и пониманием их фундаментальных ограничений.

1.3 Требования к разрабатываемому алгоритму управления рисками успешности реализации кибератак

Учитывая вышеописанные недостатки существующих алгоритмов управления рисками, можно составить список требований для алгоритма, который способен предложить лучшее решение. Разрабатываемый алгоритм должен быть:

– гибким – алгоритм не должен зацикливаться на определенных тактиках и техниках атакующего, а обеспечивать защиту на более глубоком уровне;

– актуальным – функционировать без использования базы совершенных атак;

– реалистичным – переводить технический риск в финансовый и репутационный.

2 Алгоритмы управления успешностью реализации кибератак

2.1 Алгоритм управления рисками с использованием биноминального распределения

2.1.1 Биноминальное распределение

Биноминальное распределение является одним из важнейших в статистике. Оно представляет собой дискретное распределение и возникает, по крайней мере, в ситуациях двух типов:

- делается выборка из совокупности, состоящей из элементов двух типов;
- рассматривается цепочка экспериментов, имеющие только два исхода.

Представляется серия испытаний, каждое из которых имеет два возможных исхода (успех, неуспех). Вероятность успеха p, а вероятность неуспеха $(1 - \mathrm{p})$ соответственно. Число испытаний — n. Пусть X — случайная величина равная количеству успешных испытаний в серии из n испытаний. Тогда X имеет биномиальное распределение с параметрами n и p.

Соответствующие вероятности определяются формулой Бернулли (1):

$$p_i = P_n^{x_i} = C_n^{x_i} p^{x_i} q^{n-x_i}, \quad (1)$$

где n – количество независимых испытаний;

p – вероятность появления события A в каждом испытании;

$q = 1 - p$ – вероятность непоявления события A в каждом испытании;

$x_i \in [0, n], x_i \in N$ – сколько раз может появиться событие A в данной серии испытаний (список всех возможных значений).

2.1.2 Разработка алгоритма управления рисками

Биноминальное распределение можно использовать для описания атак, реализация которых имеет исходы успех/неудача. Также их можно рассматривать,

как серию одинаковых атак через одну единственную уязвимость.

Адаптация формулы 1 под расчет вероятности наступления ущерба будет иметь вид (формула 2):

$$P(x, n, p) = C_n^x p^x (1-p)^{n-x}, x = 0, 1, 2, \dots, n \quad (2)$$

где x – число успехов;

n – количество совершенных атак;

p – вероятность успешности реализации атаки.

Число успехов – случайная величина, определяющая, сколько раз произошло событие «успех» в серии из n независимых испытаний. При обеспечении кибербезопасности данный параметр определяет число успехов при совершении атак в количестве n.

Количество совершенных атак n – величина фиксированная и зависит только от злоумышленника, так как только он решает, атаковать ему систему или нет. В случае же если атака будет проводиться, становится выбор в количестве атак, времени их исполнения, выборе целевого объекта и интенсивности. Помимо этого, при построении модели риска моделируется гипотетическое поведение злоумышленника, что важно именно для сценарного риска и определения устойчивости системы. Таким образом ввиду того, что число успехов является случайной величиной, а количество совершенных атак – параметр, зависящий от злоумышленника, можно сделать вывод, что на данные параметры специалист по ИБ повлиять не может.

Тогда биномиальное распределение изучаемой ситуации (при $n = 10, p = 0{,}5$) имеет вид, представленный на рисунке 1.

При этом распределение будет смещаться в правую или левую сторону в зависимости от изменения параметра p. При снижении этого параметра до 0,01 имеется следующее распределение, представленное на рисунке 2.

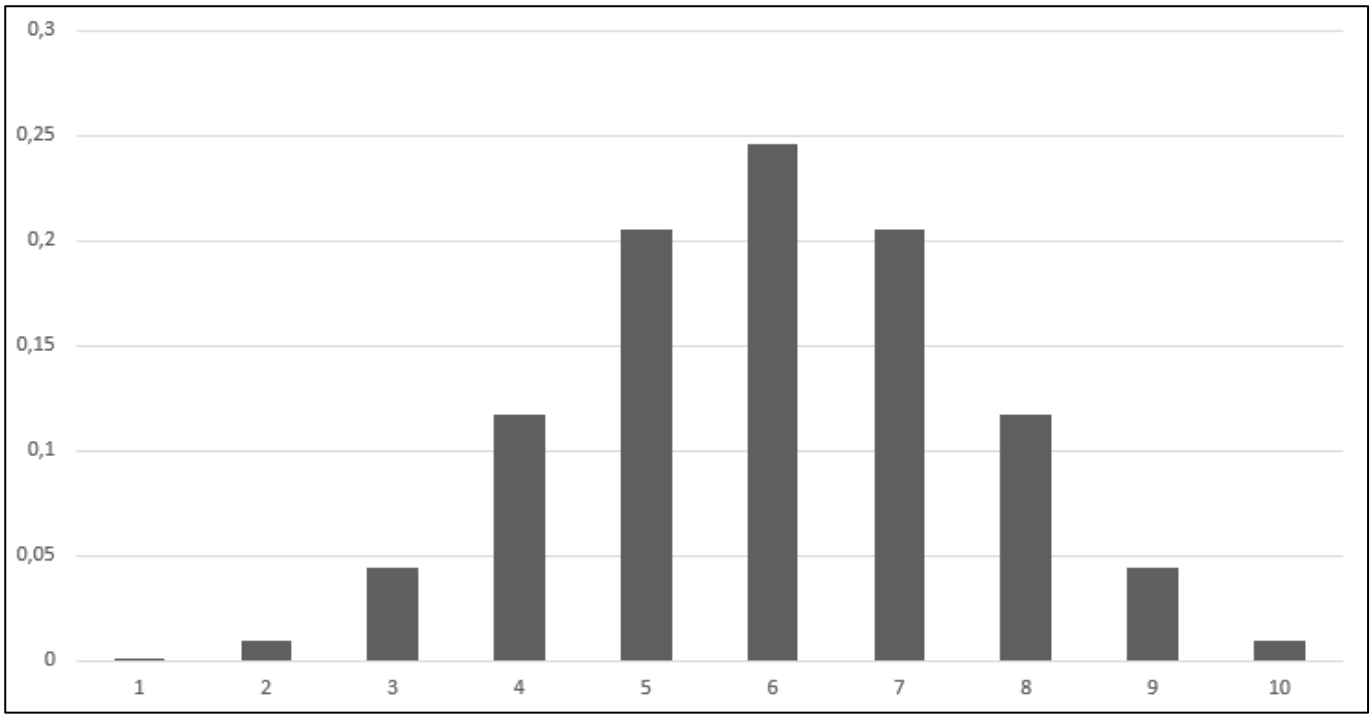


Рисунок 1 – Биномиальное распределение при $n = 10, p = 0{,}5$

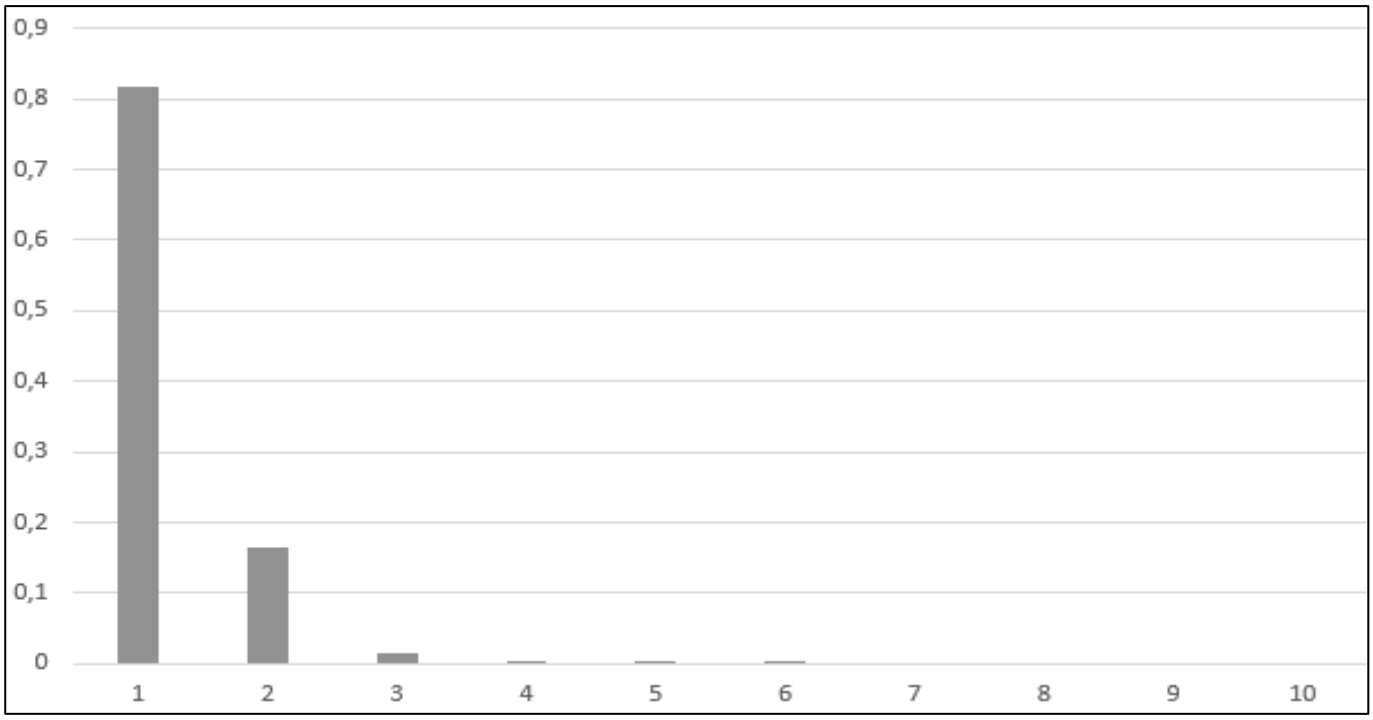


Рисунок 2 – Биномиальное распределение при $p = 0{,}01$

В противном же случае, при повышении этого параметра до практически самого максимального значения – 0,99, распределение будет иметь вид, представленный на рисунке 3.

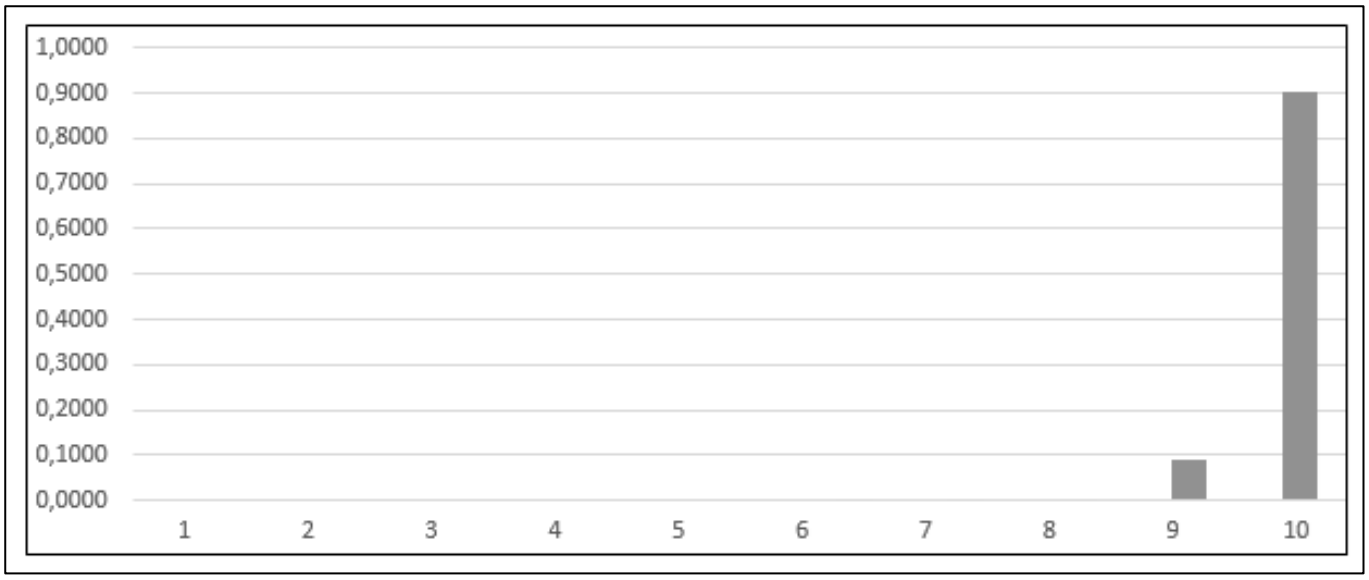


Рисунок 3 – Биномиальное распределение при $p = 0{,}99$

Определение вероятности $p_{\text{кр}}$, при которой риск в заданном диапазоне ущерба не превышает заданного уровня, позволит определить уровень, до которого нужно будет снизить исходную вероятность.

Если атака направлена на нарушение работы оборудования, кражу данных, несанкционированный доступ (НСД) к информации, то снижение вероятности целесообразно осуществлять за счёт внедрения тех или иных средств защиты информации или организационно-правовых мер. Иначе если атака носит социальный характер (фишинг), то есть направлена на человека, на побуждение его к каким-то действиям, необходимо использовать организационно-правовые меры, направленные на повышение осведомленности сотрудников и понижения соответствующей вероятности.

В рамках анализа фишинга целесообразно рассматривать последний предложенный метод, как метод регулирования рисков, поскольку такие действия являются наиболее эффективным способов борьбы подобными атаками. При этом эффективность защиты во многом зависит от качества обучения, периодичности закрепления старого материала и своевременного изучения нового, а начальная вероятность успешности атаки не будет равняться 0,5, а будет определяться ростом числа обученных сотрудников.

Согласно статистике (IBM, KnowBe4, tadviser, SecureList) 60% сотрудников способны открыть вредоносную ссылку, вложение, архив с ВПО, exe-файл или иное фишинговое сообщение, поскольку не смогли правильно идентифицировать эти письма, при этом 30% могут передать учетные данные. Однако, необходимо учитывать, что не каждый переход по ссылке приводит к компрометации, поэтому учитывается последовательность всех действия, начиная от нажатия, ввода данных. Поэтому получается, что успешность фишинг-атаки на необученного сотрудника 2,5%-5%, тогда как на обученного 0,5%-1,5%. Отсюда получаются средние значения последней статистики равные 3,5% и 1% соответственно.

Исходя из полученной статистики, вероятность успешности атаки на необученных сотрудников будет равна $p_u = 0{,}035$, а на обученных $p_t = 0{,}01$. Отсюда следует приблизительное значение, во сколько раз вероятность атаки на обученного сотрудника больше, чем на необученного (формула 3):

$$R = \frac{p_u}{p_t} = \frac{0{,}035}{0{,}01} = 3{,}5 \quad (3)$$

Получается, что вероятность успешности атаки на необученных сотрудников больше, чем на обученного в 3,5 раза.

Определение новой формы расчета вероятности, исходя из конкретной задачи – защиты от фишинга. Пусть:

N – общее число сотрудников;

x_t – количество обученных сотрудников;

p_u – вероятность успеха на необученного сотрудника;

p_u – вероятность успеха на обученного сотрудника.

Сама вероятность успеха атаки на необученного сотрудника составляет 0,5, поскольку он может как совершить интересующее злоумышленника действия, так и нет. Тогда новая вероятность, учитывающая эти параметры будет рассчитываться следующим образом по формуле 4:

$$p_y = \frac{(N - x_t)p_u + x_t p_t}{N} \quad (4)$$

Отсюда, достигая нужного значения вероятности путем обучения сотрудников, значение риска удается снизить до заданного уровня в указанном диапазоне.

Проведенный анализ позволяет сделать вывод, что вероятность успешности реализации атаки – параметр, на который может повлиять специалист ИБ, при этом данный расчет является альтернативой расчета риска при конкретном типе атаки.

При расчете риска также необходимо учитывать уязвимость, через которую будет реализована угроза. Для анализа уязвимостей используются различные метрики. Однако в контексте обеспечения информационной безопасности целесообразно рассматривать несколько механизмов документирования и оценки, позволяющих получить среднюю оценку на их основе. Таковыми инструментами являются калькуляторы CVSS, EPSS, OWASP, поскольку в отличие от иных аналогов они не являются узконаправленными. Более того они имеют понятный и удобный для пользователя интерфейс, позволяющий осуществить расчет необходимой оценки.

Исходными данными для получения финальной оценки являются оценки критичности из калькуляторов CVSS 3.1, CVSS 4.0, EPSS, OWASP. Результаты работы этих калькуляторов основаны на группе метрик, которые остаются неизменными с течением времени.

Далее необходимо нормализовать имеющиеся оценки. Сделать это можно по формуле 5:

$$s_{norm} = \frac{s - s_{min}}{s_{max} - s_{min}} \quad (5)$$

где s_{norm} – нормализованная оценка;

s_{min} – минимальная оценка в калькуляторе;

s_{max} – минимальная оценка в калькуляторе.

После этого необходимо вычислить среднеквадратическую оценку s_{final} по формуле 6. Ее целесообразно рассматривать, поскольку данный метод учитывает вариативность самих данных и влияние крайних значений.

$$s_{final} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} s_{norm.i}^2} \quad (6)$$

При использовании биномиального распределения рассматривается ситуация использования серии одинаковых кибератак через одну уязвимость, но сценарии атаки могут варьироваться в зависимости от используемой уязвимости среди возможного перечня уязвимостей, поскольку пары «вид атаки – эксплуатируемая уязвимость» формируются следующим образом:

– на основе классификации CAPEC или MITRE идентифицируется i-й вид атаки;

– на основе базы CAPEC и классификации CWE определяются ошибки, которые могут быть эксплуатированы i-м видом атаки;

– на основе базы CVE определяем уязвимости, которые соответствуют программным ошибкам;

– с помощью ресурса CISA CEV производим выборку наиболее актуальных уязвимостей;

– определение уязвимостей, характерных, для защищаемой системы, объекта, актива.

Получается, что одной программной ошибке может соответствовать несколько уязвимостей. На основе суммы среднеквадратических оценок и среднеквадратической оценки конкретной эксплуатируемой уязвимости получается вероятность эксплуатации единичной уязвимости p_{cve} (формула 7):

$$p_{cve} = \frac{S_{final}}{\sum_{i=1}^{M} S_{final}} \quad (7)$$

При расчете риска успешности реализации кибератаки целесообразно рассматривать связки: CAPEC-CWE-CVE. Это позволит оценить, насколько реалистично, что шаблон CAPEC будет использован в конкретной системе с успешным результатом.

Использование базы CAPEC для оценки риска атак позволяет перейти от оценки отдельных уязвимостей, связок CWE-CVE, к анализу шаблонов атак, отражающих поведение атакующего. База данных CAPEC позволяет обеспечить уровень обобщения, необходимый для стратегического управления рисками. Она ориентирована на тактико-технические сценарии и позволяет оценивать не только наличие уязвимости, но и вероятность её практической реализации.

Переход от шаблона атаки CAPEC к категории уязвимостей CWE позволяет связать обобщенные сценарии атак с конкретными техническими слабыми местами, которые могут быть реализованы в инфраструктуре. При рассмотрении конкретного CWE становится рассматривать связанные с ним CVE.

Для оценки вероятности успешной реализации шаблона CAPEC можно использовать среднее значение EPSS для всех связанных с CWE CVE. Оценка EPSS – это вероятность того, что уязвимость действительно будет эксплуатироваться в реальности. Она дает динамическую оценку на основе реальных данных об объективности атак и подчеркивает факт эксплуатации. Её целесообразно использовать ввиду того, что оценка CVSS v3.1, CVSS v4.0 позволяет оценивать ущерб и технические характеристики, а оценка OWASP зависит от человека и контекста и является субъективной экспертной оценкой. Отсюда получаем, что оценка EPSS – объективный индикатор «живости» шаблона атаки.

Тогда для оценки вероятности успешной реализации шаблона CAPEC можно использовать следующую формулу 8:

$$p_{capec} = \frac{1}{M}\sum_{i=1}^{M} EPSS(CVE_i) \quad (8)$$

где M – количество CVE, связанных с шаблоном CAPEC;

$EPSS(CVE_i)$ – значение EPSS для каждой уязвимости.

Ни на p_{capec}, ни на p_{cve} специалист по ИБ повлиять не может, поскольку эти вероятности получается на основе статистических данных.

Отсюда получаем, что единственный параметр, на который может повлиять специалист по ИБ является вероятность успешности реализации атаки за счет внедрения тех или иных технических, организационно-правовых, криптографических и физических мер защиты информации.

Обозначим целевую меру эффективности защиты информации, как E_k. Этот параметр может задаваться экспертно или по данным. Каждая мера защиты информации будет определяет максимальное влияние на снижение исходной вероятности, поэтому необходимо ввести вариант оценки значения этого параметра, что представлено в таблице 1.

Таблица 1 – Вариант оценки значения мер защиты информации

Качественная оценка параметров	Количественная оценка параметров
Меры умеренной эффективности	0,3
Меры средней эффективности	0,5
Высокоэффективные меры	0,9

Применение меры умеренной эффективности обеспечит заметное, но недостаточное снижение риска, что означает, что мера помогает, но не решает проблему полностью. Такими мерами являются базовое обучение, фильтры почты, двухфакторная аутентификация и другие.

Применение меры средней эффективности позволяет сказать о том, что принятая мера дает серьезную защиту, заметно снижает риск, но проблему полностью

не решает. Таковыми мерами являются стандартные правила Firewall, блокирующие половину попыток, DLP-системы базового уровня и другие.

Высокоэффективные меры почти полностью нейтрализуют конкретный вектор атаки, но риск всегда остается. Особенностью таких мер можно считать использование современных технологий, лучших тактик противоборства, регулярных тестирований и аудитов. Мерами позволяющими с высокой эффективностью снизить риск являются: использование EDR/XDR-систем, многоступенчатых систем защиты с обучением, непрерывных тестов от Red Team/Purple Team.

Таким образом, вероятность с учетом внедренных мер защиты информации получаем следующую формулу получения вероятности успешности реализации атаки (формула 9):

$$p_y = p_{\text{исх}} \cdot \prod_{i=1}^{K} (1 - E_i) \tag{9}$$

где $p_{\text{исх}}$ – исходная вероятность;

K – количество введенных мер.

В формуле используется $(1 - E_i)$, чтобы получить долю вероятности, которая остается после применения меры, то есть остаточный риск.

Может быть и обратная ситуация, когда нам необходимо повысить вероятность и увеличить риск при больших значениях ущерба. Это может использоваться в качестве управляемого эксперимента для моделирования наихудшего сценария, обоснования необходимости инвестирования средств в защиту. Поэтому активное повышение вероятности успешной реализации атаки целесообразно применять в целях пентестинга, Red Team-операций и оценки защищенности. Искусственное повышение вероятности достигается за счет снижения эффективности мер защиты или полное их отсутствие. В результате чего можем иметь следующую ситуацию (формула 10):

$$p_y = p_{\text{исх}} \tag{10}$$

Таким образом целесообразно ввести величину, включающую в себя все 3 вероятности (формула 11):

$$F_j(p_{\text{у}}, p_{cve}, p_{capec}) = p_{\text{у}} * p_{cve} * p_{capec} \tag{11}$$

Для оценки влияния защитных мер на итоговую вероятность успешности реализации атаки можно использовать градиентный подход, что позволит определить, какие меры оказывают наибольшее влияние на риск, количественно оценить их эффективность и принимать решения о приоритетах при выборе защитных стратегий.

Исходя из этого, становится целесообразным использование принципа обратной связи. Задача представляет собой стохастическую схему, где возмущение по «Основам теории управления» – атака с заданной вероятностью успеха. Управляющим воздействием u_t, изменяемым на основе текущего состояния системы является средство управления. Модель возмущений – сам случайный процесс. Его целесообразно использовать, поскольку принцип обратной связи предполагает постоянное обновление u_t на основе текущей ошибки управления.

Система мониторинга считает уровень риска и определяет до какого уровня необходимо понизить вероятность, чтобы в заданном диапазоне ущерба уровень риска не превышал допустимого уровня. Алгоритм управления усиливает меры, внедряя их в систему.

Управляющим воздействием является вектор $u_j = \{u_{1,t}, u_{2,t}, \dots, u_{k,t}\}$, где $u_{j,t} \in [0, E], j = 1 \dots K$, который будет влиять на вероятность. При этом сами меры носят временный характер, теряя свою эффективность, устаревая, с каждым некоторым шагом. В рамках регулирования рисков и определения наиболее подходящего варианта параметров целесообразно регулировать эти параметры, чтобы увидеть, как меняется вероятность. Выбор мер основывается на их

эффективности и имеющемся бюджете на их реализацию, поскольку цена на разрабатываемые средства растет, как и спрос на более современное оборудование. Имея неограниченный бюджет, защита организаций достигла бы беспрецедентного уровня, однако абсолютной безопасности даже в этом случае существовать не может. Организация может получить идеальную инфраструктурную безопасность, сверхсовременные технологии, человеческий фактор под контролем благодаря постоянному обучению, своевременному отсеву сотрудников с высоким риском инсайдерских угроз, но 100% безопасности мешает тот же самый человеческий фактор и атаки типа «социальная инженерия», уязвимости нулевого дня и иные угрозы.

Вводится также целевое значение функции $F_{\text{цел}}$, которого необходимо добиться, чтобы риск был меньше заданного уровня в нужном диапазоне ущерба.

Вычисляем ошибку управления (ошибку обратной связи) на некотором шаге e_t по формуле 12:

$$e_t = F_{\text{цел}} - F_j\left(p_{\text{у}}, p_{cve}, p_{capec}\right) \tag{12}$$

Ошибка управления позволяет понять, насколько уровень вероятности, рассчитанной по функции, отклоняется от целевого значения, при котором будет достигаться необходимый уровень риска в нужном диапазоне ущерба.

Если $e_t > 0$, значит нужно усиливать меры защиты и увеличивать $u_{j,t}$, если наоборот, то дальнейшего усиления не требуется. Поэтому исходя из этой ошибки обновляем управляющие воздействия $u_{j,t}$ по формуле 13:

$$u_{i,t+1} = u_{i,t} + e_t \cdot F_j \cdot \eta \tag{13}$$

где η – шаг обратной связи, который может принимать как фиксированное значение, так и переменное в зависимости от выбранной стратегии управления, поскольку если система работает в относительно стабильных условиях, можно задать фиксированное значение, если же важна точность или динамика среды быстро

меняется, лучше использовать адаптивный вариант. Может принимать значения в диапазоне (0;1). Если шаг слишком большой – может возникнуть перерегулирование, в противном случае система будет медленно реагировать.

В данной формуле используются частные производные, поскольку $u_{j,t}$ является управляющим воздействием, влияющим на вероятность и соответственно на риск, а F_j - функция, зависящая от нескольких вероятностей. Поэтому функция F_j зависит от $u_{j,t}$.

Целью этих вычислений становится следующий результат (формула 14):

$$F_j\left(p_{\text{у}}, p_{cve}, p_{capec}\right) \rightarrow F_{\text{цел}} \tag{14}$$

Полученные формулы позволят моделировать изменение риска за шаг t (дискретное обновление, поскольку с помощью дискретного распределения происходит расчет успехов – целое число.

Данный расчет может осуществляться в отношении конкретного объекта, поскольку каждая огибающая распределения вероятностей моделирует риск, связанный с ним (уязвимостью, компонентом системы, сценарием атаки). В качестве объекта целесообразно рассматривать уязвимость, поскольку расчет идет с учетом её параметров и сама угроза может реализоваться за счет этой уязвимости. Получаем, что каждому объекту соответствует своя огибающая риска, рассчитанная через биномиальное распределение, зависящая от индивидуальных параметров: числа атак, вероятности реализации, уровня воздействия. Комплексную оценку риска при совмещении нескольких техник или угроз можно осуществить в двух случаях: при наступлении независимых событий и зависимых (случай, когда ущерб, наносимый при реализации второй угрозы, зависит от последствий реализации первой).

В первом случае события независимы, тогда суммирование вероятностей реализации угроз по нескольким объектам обосновано тем, что каждый объект представляет собой отдельный вектор риска. Поскольку риски могут реализовываться

независимо с возможной последующей нормировкой для сохранения вероятностной интерпретации.

Внедряя ущерб x, можно оценить риск следующим образом (формула 15):

$$Risk(x, n, F_j) = x * C_n^x p^x (1 - F_j)^{n-x} \quad (15)$$

Однако в этом случае мы можем иметь большой разброс значений рисков как на рисунке 4.

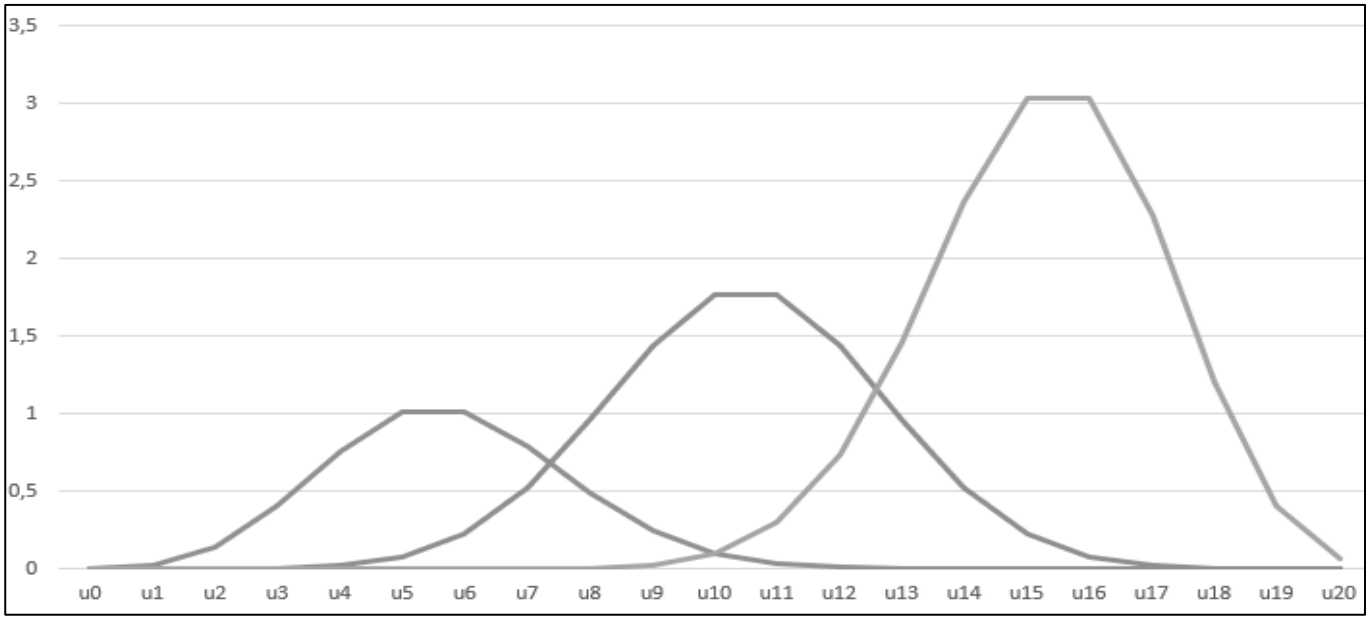


Рисунок 4 – Огибающие распределений с учетом ущерба x

Вместе с тем, возможно реализовать управление с целью уравнивания этих рисков по их максимальным величинам, поскольку считать параметр x ущербом возможно только в том случае, если ценность информационного ресурса всякого компонента равновелики. Отсюда ущерб равен $x \cdot C_i$, где C_i – ценность защищаемого ресурса i-ой компоненты. Следуя этому свойству, целесообразно перераспределить ценность ресурса системы с учетом максимумов риска компонентов без учета этих ресурсов.

Расчет C_i осуществляется по формуле 16:

$$C_i = \frac{1}{p_i k_i} \frac{1}{\sum_{j=1}^{L} \frac{1}{p_j k_j}} \tag{16}$$

Отсюда стоит добиваться равенства (формула 17):

$$\max\left(F_{j_1} \cdot x_1 \cdot C_1\right) = \ldots = \max\left(pF_{j_L} \cdot x_L \cdot C_L\right) \tag{17}$$

После чего сложив несколько огибающих и перераспределяя C_i, получается следующий результат, представленный на рисунке 5 (при параметрах $n = 20, p1 = 0{,}25, p2 = 0{,}5;\ p3 = 0{,}75$).

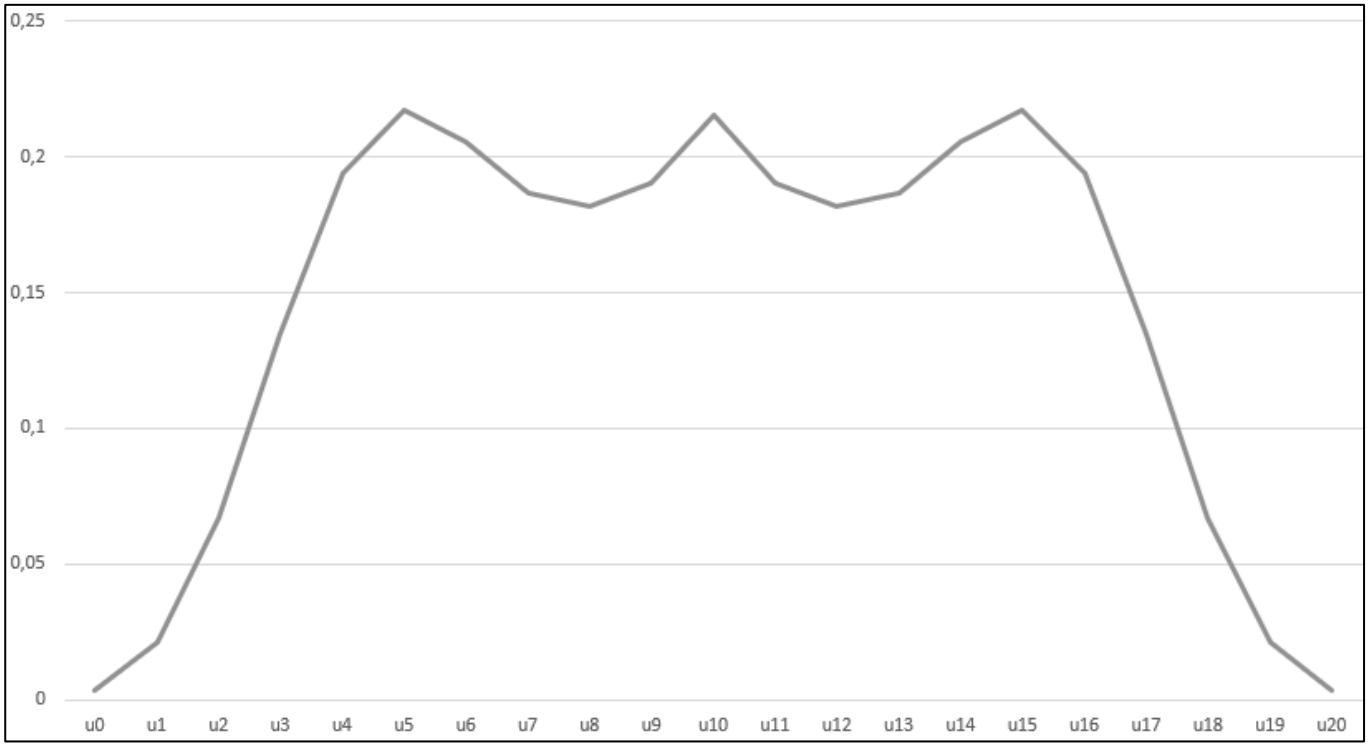


Рисунок 5 – Огибающая суммы

Графически итоговый результат с рисунка 5 можно получить из графиков, как представлено на рисунке 6 при тех же параметрах.

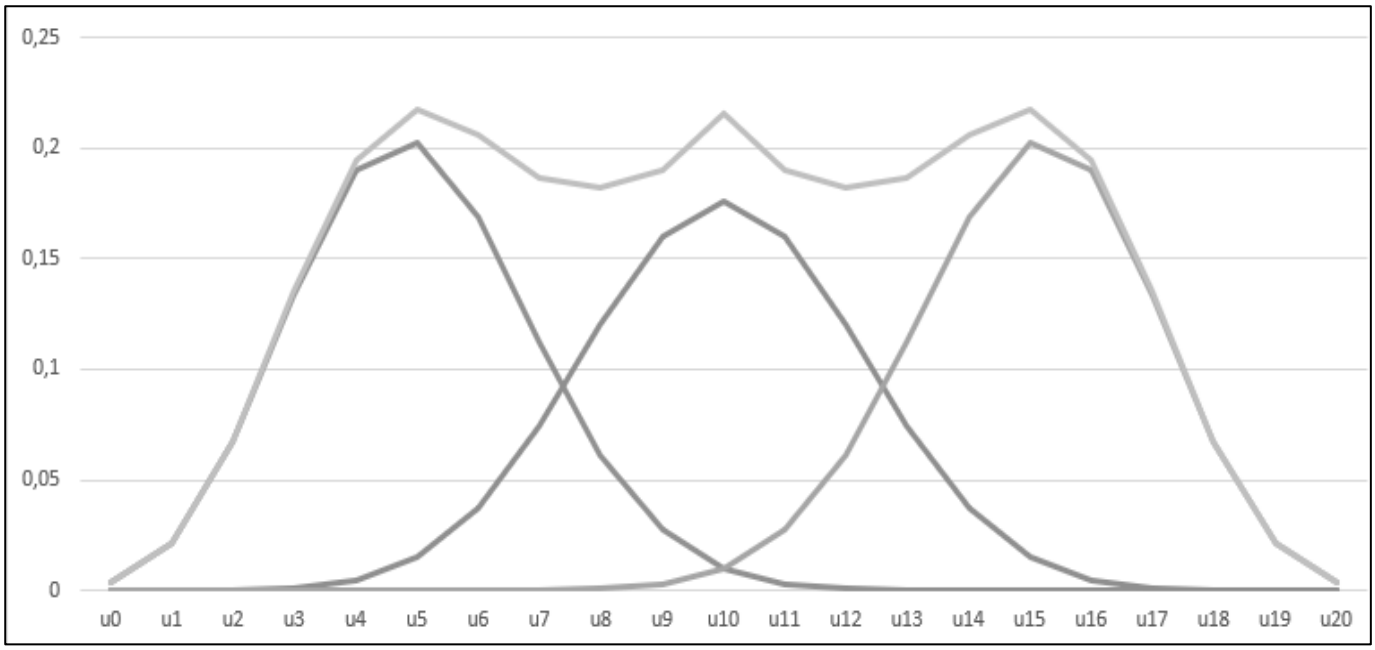


Рисунок 6 – Графики отдельных огибающих и суммарный

Это достигается за счет следующей формулы 18:

$$Risk_{total} = \sum_{j=1}^{L} Risk_j \tag{18}$$

где L – количество объектов;

P_{total} – суммарный риск по всем объектам.

Однако при таком расчете получается, что сумма рисков превышает 1, чего не может быть, поэтому чтобы сохранить интерпретацию результата как вероятностного распределения, его необходимо нормировать. Поэтому получается следующая формулу (19):

$$Risk_{total}^{norm} = \frac{1}{L}\sum_{k=1}^{L} Risk_j \tag{19}$$

Тогда получается следующий нормированный график распределения (рисунок 7).

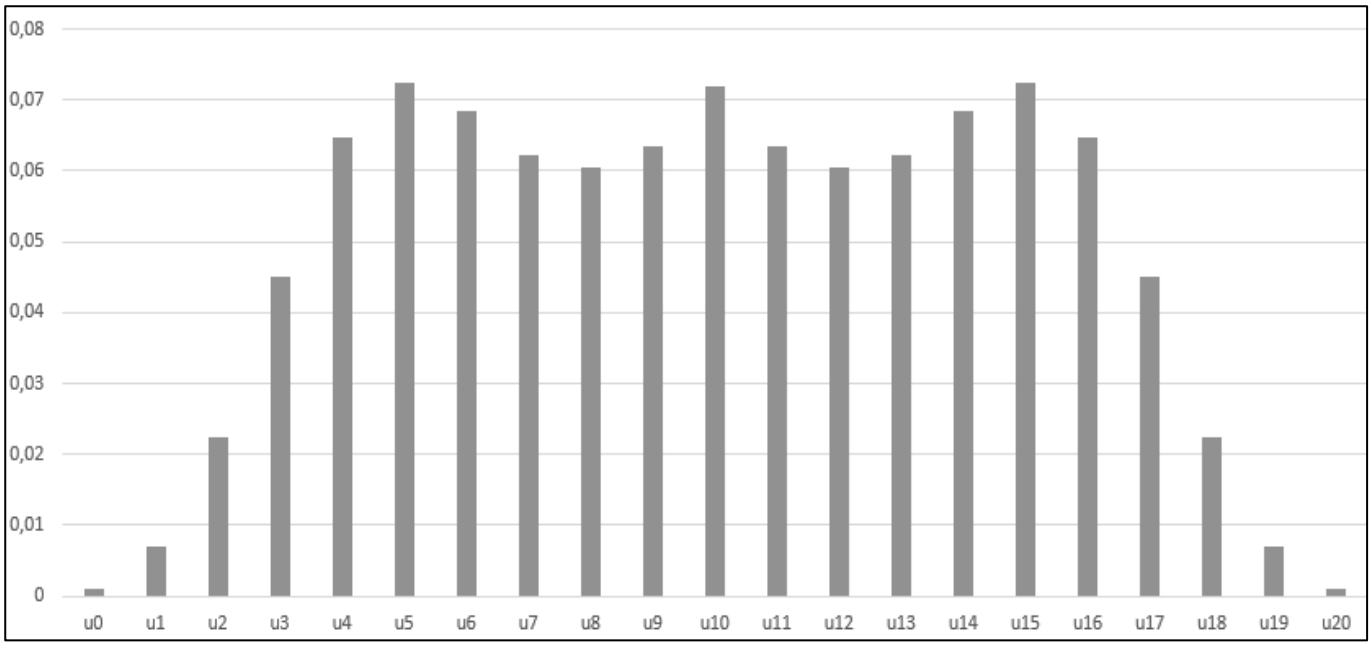


Рисунок 7 – Нормированное распределение суммы вероятностей

2.1.3 Последовательность выполнения действий

Алгоритм управления рисками успешности реализации кибератак с использованием гипергеометрического распределения можно представить следующим образом (см. рис. 8):

1) Первоначально определяются значения следующих параметров:
   - число успехов x;
   - количество совершенных атак n;
   - вероятность успешности реализации атаки p;
2) Проведя аудит системы информационной безопасности необходимо определить, какие меры защиты информации были внедрены и на основе полученных данных рассчитать по формуле 20 вероятность успешности реализации атаки;

$$p_y = p_{\text{исх}} \cdot \prod_{i=1}^{K} \left(1 - u_i(t)\right) \tag{20}$$

3) Следующим этапом необходимо определить уязвимость, через которую, наиболее вероятно, будет совершена атака. Для этой уязвимости, используя оценки критичности из калькуляторов CVSS 3.1, CVSS 4.0, EPSS и OWASP необходимо рассчитать финальную оценку путем нормализации и вычисления среднеквадратичной оценки по формулам 5 и 6 соответственно. А на основе этих данных вычисляется вероятность эксплуатации единичной уязвимости (формула 21);

$$p_{cve} = \frac{s_{final}}{\sum_{i=1}^{M} s_{final}} \tag{22}$$

4) Так как при расчете риска успешности реализации кибератак имеет место рассматривать связки CAPEC-CWE-CVE, то на данном этапе осуществляется переход от оценок CVE к значению вероятности успешной реализации шаблона CAPEC путём нормализации оценок CVE уязвимостей, связанных с определенным шаблоном CAPEC через CWE. Данное значение вычисляется по формуле 23:

$$p_{capec} = \frac{1}{M} \sum_{i=1}^{M} EPSS(CVE_i) \tag{23}$$

5) Следующим этапом по формуле 24 вычисляется величина F_j, которая включает в себя все 3 вероятности, которые были рассчитаны на предыдущих шагах алгоритма.

$$F_j(p_{\text{у}}, p_{cve}, p_{capec}) = p_{\text{у}} \cdot p_{cve} \cdot p_{capec} \tag{24}$$

6) Далее по формуле 25 необходимо вычислить ошибку управления, которая покажет, нужно ли прикладывать усилия для достижения заданного уровня риска F_j.

$$e_t = F_{\text{цел}} - F_j(p_{\text{у}}, p_{cve}, p_{capec}) \tag{25}$$

7) Если полученное значение $e_t > 0$, значит меры защиты нуждаются в обновлении или усилении, а значит необходимо получить новое значение управляющего воздействия на систему $u_{i,t}$. Оно рассчитывается по формуле 26:

$$u_{i,t+1} = u_{i,t} + e_t \cdot F_j \cdot \eta \tag{26}$$

8) Цель вышеуказанных вычислений – добиться целевого значения итоговой вероятности (формула 27):

$$F_j(p_y, p_{cve}, p_{capec}) \to F_{\text{цел}} \tag{26}$$

9) Следующим этапом необходимо оценить ущерб x, который может наступить в случае успешной реализации атаки злоумышленником. С учетом ущерба, по формуле 28 вычисляется значение риска, которое не должно превышать заданного.

$$Risk(x, n, F_j) = x \cdot C_n^x p^x (1 - F_j)^{n-x} \tag{28}$$

10) Если же таким способом добиться необходимого значения риска не удалось, то имеет смысл использовать такой параметр, как ценность ресурса C_i. Тогда, используя совокупность параметров необходимо добиваться следующего равенства (формула 29):

$$\max(p_1 \cdot x_1 \cdot C_1) = \ldots = \max(p_L \cdot x_L \cdot C_L) \tag{29}$$

11) Как только равенство на предыдущем шаге будет удовлетворено, по формуле 30 проводится суммирование и нормализация значений риска для получения его визуализации.

$$Risk_{total}^{norm} = \frac{1}{L} \sum_{k=1}^{L} Risk_j \tag{30}$$

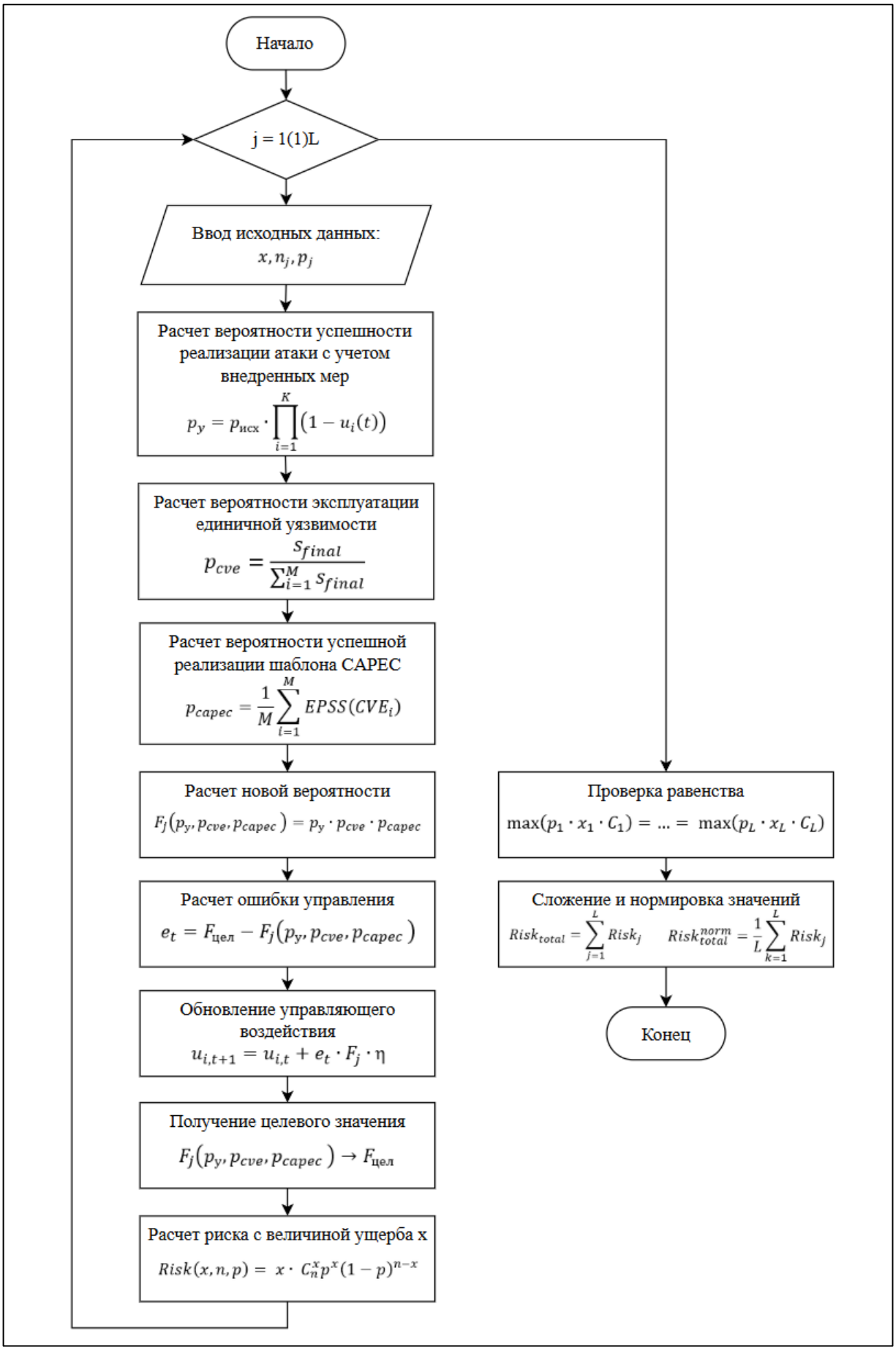


Рисунок 8 – Общий вид алгоритма

2.1.4 Область применения алгоритма

Разработанный алгоритм отлично подходит для управления рисками успешности реализации кибератак. Он позволяет просчитать риск для любых уязвимостей и шаблонов атак, однако от этого требует статистических данных, таких как метрики уязвимостей и число «успехов» атак. Наличие в расчетах данных, которые можно получить только путем сбора статистики всегда ограничивает универсальность алгоритма.

В связи с особенностями вычислительной части алгоритма, его специфика – фишинговые атаки. Пресечение таких атак, безусловно, важно и необходимо, ведь фишинг является одним из простейших методов для злоумышленника получить доступ к защищаемой информации. Однако узкая специализация алгоритма на конкретном виде атак говорит о том, что использование его для вычислений и регулирования рисков других видов атак может привести к получению некорректных данных.

2.2 Алгоритм управления рисками с использованием гипергеометрического распределения

2.2.1 Гипергеометрическое распределение

Гипергеометрическое распределение описывает ситуацию, когда, например, из коробки, в которой находится K белых и N черных шаров, вынимают наудачу n шаров, при этом не возвращая их обратно и так, чтобы количество вынутых шаров не превышало общего количества шаров в коробке, то есть, чтобы выполнялось условие $n \leq N$. В данном случае, термин «наудачу» означает, что появление любого набора черных и белых шаров размером n имеет одинаковую вероятность [11].

При решении данной задачи следует учитывать, что при следующих условиях на формуле (1):

$$k > K,$$
$$n - k > N - K, \quad (1)$$

искомая вероятность будет равна нулю, так как соответствующее событие будет невозможно. Соответственно, в таком случае необходимо данное условие прописать по умолчанию для гипергеометрического распределения. Получим (2):

$$k \leq K,$$
$$n - k \leq N - K, \quad (2)$$

где k – количество извлеченных белых шаров;

K – количество белых шаров;

N – общее количество шаров;

n – количество извлеченных шаров.

Общее число элементарных исходов будет равняться количеству n-элементных подмножеств множества, состоящего из N элементов, и будет равняться C_N^n. Допустим, необходимо найти вероятность события A_k. Ему способствует появление любого набора, содержащего k белых шаров и $n - k$ черных. Число благоприятных исходов, согласно теореме о перемножении шансов, равно произведению количества способов выбрать k белых шаров из K и количества способов выбрать $n - k$ черных шаров из множества $N - K$. Отсюда получаем формулу (3):

$$|A_k| = C_K^k \cdot C_{N-K}^{n-k}. \quad (3)$$

Тогда вероятность события A_k будет равняться частному формулы у и общего количества элементарных исходов. Учитывая условия из формулы 2, гипергеометрическое распределение имеет вид (4) [11]:

$$P(A) = \frac{C_K^k \cdot C_{N-K}^{n-k}}{C_N^n},$$
$$k \leq K, n - k \leq N - K. \quad (4)$$

2.2.2 Разработка алгоритма управления рисками

Адаптируем определение гипергеометрического распределения под контекст темы. В таком случае получим формулу (5).

$$p(x; n, k, m) = \frac{C_k^x \cdot C_{n-k}^{m-x}}{C_n^m}, \quad x \leq k, m - x \leq n - k, \tag{5}$$

где x – число успешных атак на критичные уязвимости в системе;

n – общее число уязвимостей;

k – число критичных уязвимостей;

m – число совершенных атак, задействующих какую-либо уязвимость.

Что касательно ограничений гипергеометрического распределения, то они будут интерпретированы следующим образом: условие $x \leq k$ показывает, что злоумышленник не может реализовать в своей атаке большее количество уязвимостей, чем количество совершенных атак; а условие $m - x \leq n - k$ задает требование, по которому число атак на некритические уязвимости не может превышать их число.

Переходя к управлению рисками успешности кибератак следует рассмотреть переменные более детально, чтобы понять какие из них «управляемые», а какие – нет.

Число успешных атак на критичные уязвимости x – величина непредсказуемая и случайная. Повлиять на неё специалист по информационной безопасности не может, как и на переменную m, которая полностью зависит от злоумышленника и только от него.

Общее число уязвимостей n определяется исходным состоянием системы, обычно она зафиксирована архитектурой системы и меняет своё значение в зависимости от установленного программного обеспечения. Изменить данную величину в меньшую сторону проблематично, так как это требует большого количества различных ресурсов. Среди них:

а) затраты времени на анализ системы и тестирование её на устойчивость;

б) наличие качественных специалистов в области ИБ;

в) финансовые ресурсы на обновление системы, закупку и установку средств защиты информации

При этом с пунктом «б» в последнее время вопрос обстоит наиболее остро: по данным из отчета ISC2 только в период с 2022 по 2023 дефицит специалистов в области ИБ вырос на 12,6% и достиг 4 миллионов сотрудников. Учитывая, что за последние года дефицит только растёт – данная проблема приобретает очень внушительный характер [12, 13].

Даже если предположить, что организация имеет ресурсы для устранения существующих уязвимостей и инициирует этот процесс, то нет никаких гарантий, что на следующий день для используемого ПО выйдет обновление, несущее в себе новые уязвимости, которые могу оказаться даже серьёзнее предыдущих. При этом грамотные ИТ специалисты организации принимают за правило установку вышедших обновлений в обязательном порядке, а злоумышленнику для нахождения новых уязвимостей, зачастую, требуется некоторое количество времени. Такое правило гарантирует постоянное наличие уязвимостей в ПО на рабочих станциях информационной системы.

Соответственно, проанализировав параметры распределения, специалисту по ИБ остаётся доступным для управления только параметр k. Количество критичных уязвимостей, которое характеризует этот параметр, проще поддаётся изменению в отличие от параметра m, т.к. требует точечного подхода, в результате которого возможно устранить конкретную уязвимость, затруднить её исполнение или снизить её доступность путём удаления ПО в котором она содержится, отключения конкретных служб и сервисов ОС, сегментации сети или ограничения сетевого доступа к внутренней сети. Таким образом, уязвимость хотя всё ещё и будет доступна, но уже будет исключена из атакуемого множества и не будет входить в число n.

Также не стоит забывать про ресурсы, затрачиваемые на устранение уязвимостей. Зачастую они ограничены, поэтому с целью приближения алгоритма

регулирования к реальному процессу будет введено понятие энергии управления, как агрегированной меры на реализацию управляющих воздействий.

Энергия управления E – мера затрат ресурсов, необходимых для приведения состояния системы к заданному. В контексте управления рисками успешности реализации кибератак необходимо привести исходную систему к состоянию, при котором уровень риска в заданном диапазоне значений не будет превышать допустимого. В условиях ограниченных ресурсов (время, квалификация и количество специалистов по ИБ) следует учитывать ограничение управляющего воздействия, которое позволит сохранить стабильность работы системы ввиду отсутствия сильной и частой её перегрузки. Это ограничение представлено формулой (6).

$$\sum_{t=0}^{T} |u_t| \leq E_{max}, \tag{6}$$

где u_t – величина управляющего воздействия на шаге времени t.

Для избежания перерасхода энергии необходимо контролировать ожидаемый остаток энергии на каждом шаге. Так оставшаяся энергия $E_{\text{ост}}$ будет вычисляться по формуле (7):

$$E_{\text{ост}} = E_{max} - \sum_{t=0}^{T} |u_t| \tag{7}$$

Изменения одного параметра для адекватного регулирования риска будет недостаточно, поэтому имеет смысл ввести и учесть при расчете риска дополнительные переменные. Одними из таких будут прирост числа критичных уязвимостей λ_t и снижение их числа μ_t. Эти величины будут неизменны в процессе эксплуатации защищенной информационной системы. Их введение обусловлено тем, что при построении системы защиты используется формальная экстраполяция, то

есть предположение о сохранении в будущем прошлый и настоящих тенденций развития системы и изменение количества её уязвимостей [14].

Параметры λ_t и μ_t можно представить следующим образом (8, 9):

$$\lambda_t \approx \Delta k_t^+,\ \lambda_t \in [0, n - k_t] \tag{8}$$

$$\mu_t \approx \Delta k_t^-,\ \mu_t \in [0, k_t] \tag{9}$$

Получим, что λ_t и характеризуется как положительный прирост количества уязвимостей Δk_t^+ за шаг времени t, и максимум её достигается в тот момент, когда все имеющиеся оставшиеся уязвимости из общего числа становятся критичными на текущем шаге. А, соответственно, μ_t – количество устраненных уязвимостей Δk_t^- за шаг времени t, и её максимум означает, что все текущие критичные уязвимости устраняются без стороннего управления, так как данные равенства не учитывают управляющее воздействие u_t.

Реализовать смещение графика риска в меньшую или большую сторону можно как раз таки с учетом изменения значений переменных, указывающих на изменение количества критичных уязвимостей. С учетом этих параметров и параметра управляющего воздействия u_t получено следующее выражение (10):

$$k_{t+1} = k_t + \lambda_t \cdot u_t - \mu_t \cdot u_t = k_t + u_t(\lambda_t - \mu_t),\ \ u_t \in [0, 1] \tag{10}$$

Как указано в формуле (9) значение управляющего воздействия лежит в диапазоне $u_t \in [0, 1]$, где $u_t = 0$ означает, что управление отсутствует, т.е. система развивается естественным образом, а значит никакого ускоренного подавления уязвимостей нет. Значение $u_t = 1$ выражает наличие максимального управляющего воздействия на систему – все ресурсы обращены на контроль и устранение уязвимостей.

Если существует необходимость уменьшать значение k_t для увеличения уровня риска при меньших значениях ущерба, необходимо, чтобы число критичных

уязвимостей снижалось, что может осуществляться за счет следующих действий: осуществление активного управления u_t за счет контроля критичных уязвимостей; прирост числа критичных уязвимостей λ_t необходимо стараться держать низким за счет предусмотрительных мер, предиктивного анализа; рост значения параметра μ_t будет только способствовать снижению k_t.

Таким образом, теперь имеется два контролируемых параметра u_t и μ_t, которые и внося основной вклад в снижение риска, компенсируя λ_t.

Однако, имеет место быть и обратная ситуация: контролируемое увеличение значения параметра k_t, для того, чтобы перераспределить риск и сделать его менее концентрированным, тем самым размазать пик графика риска. В таком случае необходимы следующие действия над параметрами: целенаправленное контролируемое увеличение k_t для уменьшения концентрации риска; целенаправленное временное ослабление управляющего воздействия u_t; при этом прирост числа критичных уязвимостей λ_t обуславливается обновлением ПО, внедрением нового кода, а параметр μ_t стабильно мал и не мешает росту k_t.

Далее целесообразно будет связать управляющее воздействие u_t с параметрами положительного и отрицательного изменения количества критичных уязвимостей. Для этого необходимо ввести коэффициенты чувствительности управления к интенсивностям $\alpha, \beta \in [0,1]$. Это коэффициенты определяются априорно для различных классов систем и меняются редко и сложно. Получается, что регулирование λ_t, μ_t, u_t позволяет регулировать итоговую переменную k_{t+1}. Отсюда следует формула для расчета k_{t+1} (11):

$$k_{t+1} = \lfloor \lambda_t \cdot (1 - \alpha u_t) \rfloor - \lfloor \mu_t \cdot (1 + \beta u_t) \rfloor. \quad (11)$$

Эта формула (11) показывает, что чем больше управляющее воздействие на систему, тем меньше теп появления уязвимостей, а также больше темп устранения этих уязвимостей.

При этом же если $\alpha = 0$, то даже при максимальном управлении равном $u_t = 1$ никакого влияния на появление уязвимостей нет. А если же $\alpha_t = 1$ и $u_t = 1$, то прирост уязвимостей может быть полностью подавлен. Таким образом управляющее воздействие способствует понижению интенсивности появления уязвимостей.

В случае, если $\beta = 0$, управляющее воздействие не ускорит устранение уязвимостей. Если же $\beta = 1$ при $u_t = 1$, то снижение удваивается. Так управляющее воздействие увеличивает скорость устранения уязвимостей.

Если в рамках алгоритма, рассматривающего применение биноминального распределения, было целесообразно рассматривать отдельные уязвимости в качестве объектов, то в случае с гипергеометрическим распределением систему стоит рассматривать как набор её сегментов, содержащих уязвимости.

Данные сегменты можно считать независимыми друг от друга, поскольку каждый сегмент может обслуживать разные процессы, сервисы и отделы, иметь свой набор уязвимостей. Если атака на один сегмент не влияет напрямую на другой, то события (согласно теории вероятностей) можно считать независимыми. Внутренние политики также могу делать сегменты условно-изолированными. Поэтому, целесообразно агрегировать риск по нескольким объектам или сегментам. Однако если появятся взаимосвязи между сегментами (атака на один сегмент открывает возможность для атаки на другой), то целесообразно переходить к совместным распределениям или моделям с зависимостями, поскольку простое сложение не будет отражать реальную картину.

Допустим, что x – величина нормированного ущерба, тогда риск можно вычислить по формуле (12):

$$Risk = x\frac{C_k^x C_{n-k}^{m-x}}{C_n^m}, \tag{12}$$

где $x = 1(1)10$, что значит $x \in [1, 10]$ и при этом $x \in Z$.

Однако, в таком случае разброс значений рисков будет достаточно большим, что наглядно видно на рисунке 9.

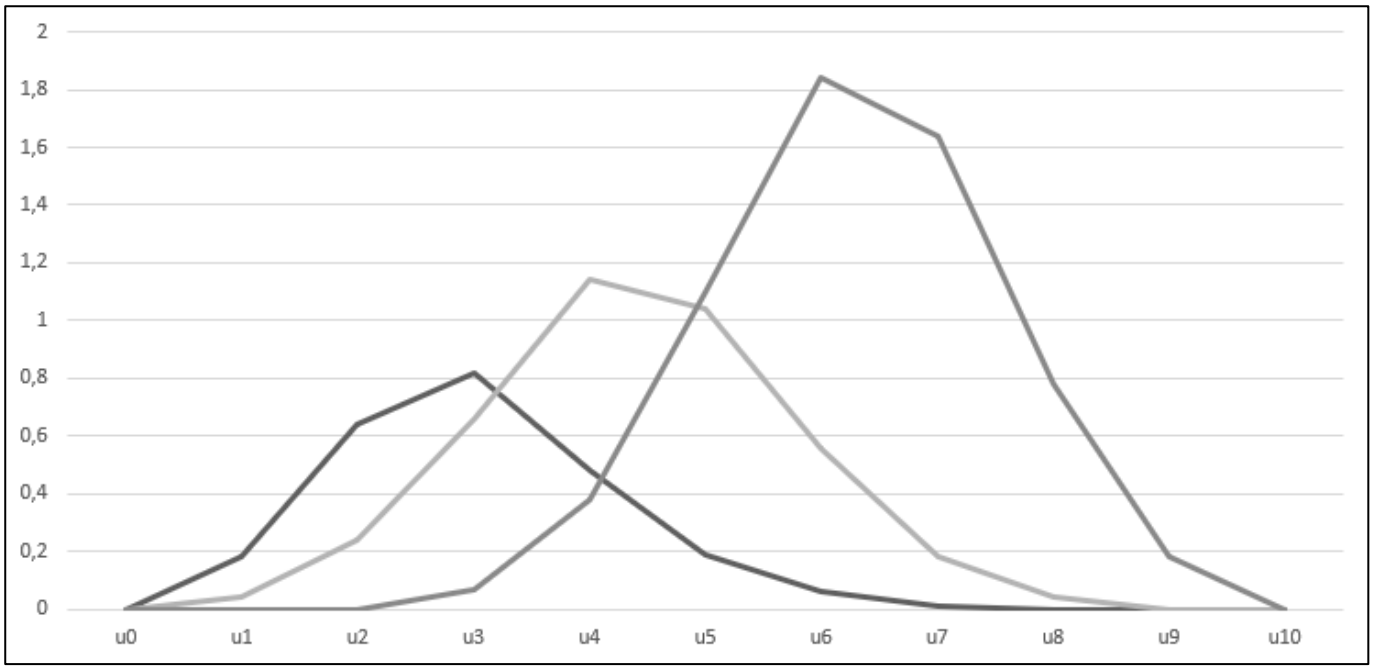


Рисунок 9 – Огибающие распределений с учетом ущерба x

Введем дополнительную величину α_i, которая будет отвечать за долю потерь защищаемого сегмента. Этот параметр будет служить локальным параметром чувствительности, указывающий на специфику отдельно взятого сегмента, что также позволит решить проблему диспропорции в системе обеспечения безопасности.

Тогда ущерб будет равен $x \cdot \alpha_i$. При перераспределении доли потерь можно оценить влияние каждого сегмента на суммарный риск. После перераспределения ущерба огибающие примут вид, отображенный на рисунке 10.

При работе с суммарным риском целесообразно ввести дополнительный параметр C_i, который будет указывать на ценность ресурса, затрагиваемую x_i успехом. Перераспределяя этот параметр, огибающие примут необходимый вид.

В таком случае суммарный риск будет рассчитываться по формуле (13):

$$Risk_{sum} = \sum_{j=1}^{m} \left(C_j \sum_{i=1}^{L} Risk_{ij} \right), \tag{13}$$

где L – количество сегментов (объектов) сети.

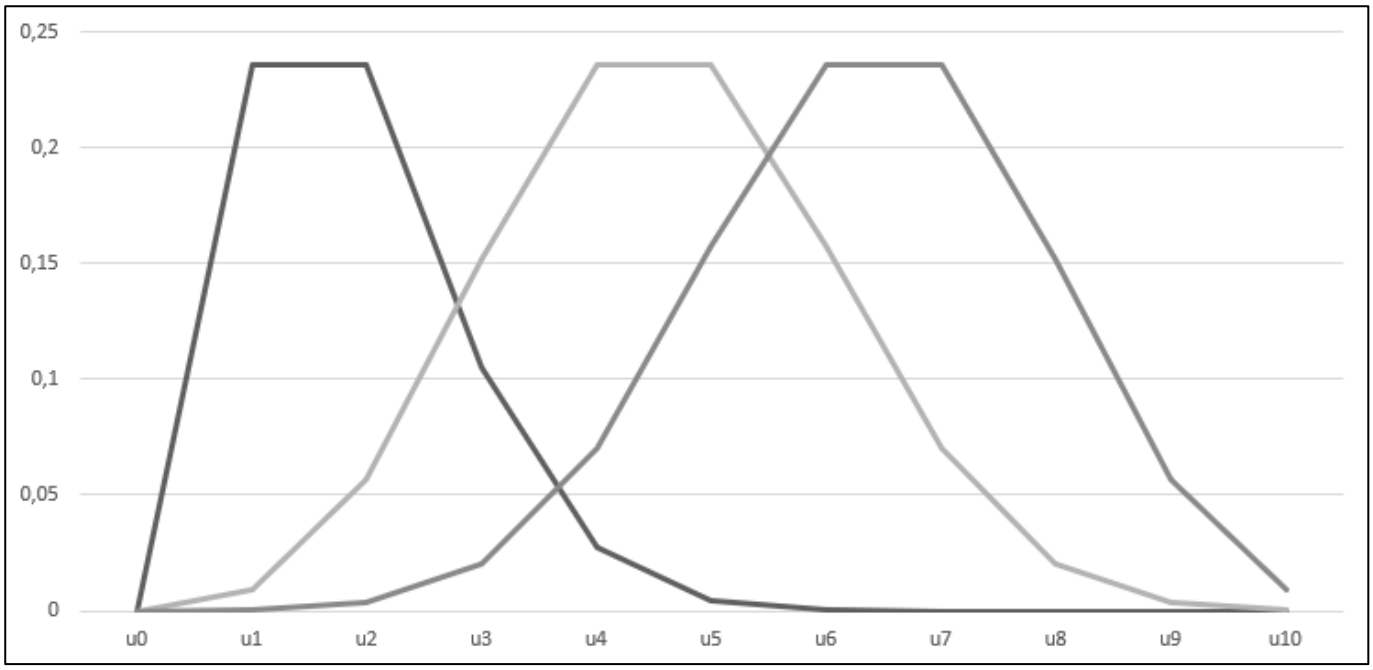


Рисунок 10 – Огибающие распределений с учетом ущерба $x \cdot \alpha_i$

Итоговый результат в графическом представлении показан на рисунке 11.

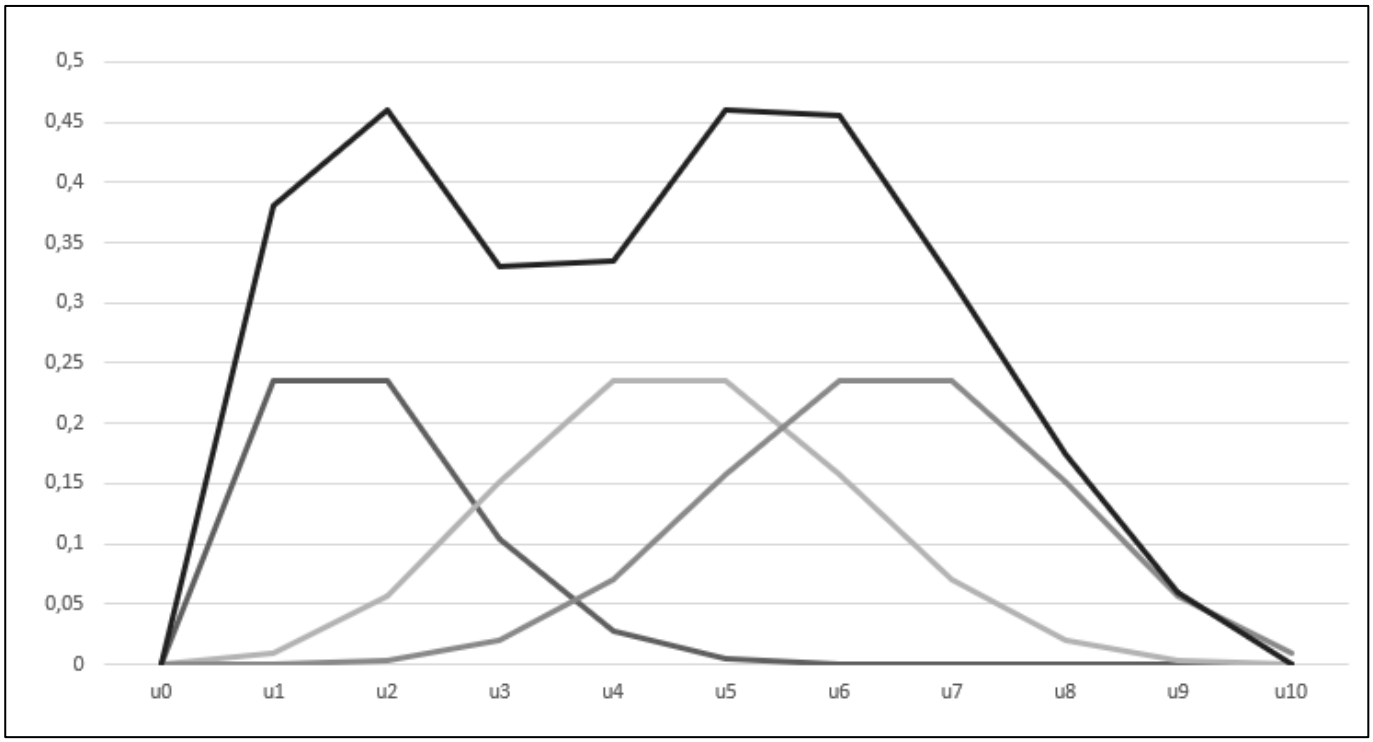


Рисунок 11 – Огибающие распределений и их суммарная

Однако при таком расчете сумма вероятностей превышает 1, чего быть не может, поэтому для сохранения интерпретации результата как вероятностного распределения, его необходимо нормировать. В связи с этим, расчеты стоит проводить по формуле (14):

$$Risk_{sum}^{norm} = \frac{1}{L}\sum_{z=1}^{L} Risk_z \quad (13)$$

Нормировка позволяет корректно сравнить риски различных событий и сделать осмысленные выводы. Более того, если вероятности не нормированы, ожидаемый риск может быть завышен или занижен из-за некорректных весовых коэффициентов. Многие математические методы предполагают работу с нормированными распределениями. Ненормированные данные могут привести к численной неустойчивости или ошибкам в последующих расчетах.

Таким образом имеется последовательность действий с использованием принципа обратной связи, основанная на динамическом обновлении параметра k_t на каждом шаге с учетом текущего состояния системы (k_t), положительного и отрицательного прироста количества уязвимостей (λ_t, μ_t), управляющих воздействий (u_t), которые и регулируют данные величины. Такой принцип позволяет регулировать риск и динамически балансировать систему в реальном времени.

Предложенный подход наиболее близок к реальным условиям функционирования информационных систем, где уязвимости могут устраняться и прибавляться, а меры защиты адаптируются к изменяющимся угрозам.

2.2.3 Последовательность выполнения действий

Алгоритм управления рисками успешности реализации кибератак с использованием гипергеометрического распределения можно представить следующим образом (см. рис. 12):

1) Ввод исходных данных для каждого из сегментов (объектов) системы. Такими объектами могут быть как филиалы организации, подсети, офисы отделов, так и отдельно взятые компьютерные устройства. На этом этапе вводятся такие данные, как:

- число успешных атак на критичные уязвимости x;
- число совершенных атак, задействующих какую-либо уязвимость m;
- число критичных уязвимостей k;
- общее число уязвимостей n;

2) Следующим этапом является расчет суммы управлений, приложенных на каждом из шагов. Она вычисляется по формуле (14). Тут не стоит забывать про ограничение энергии управления, ведь каждое выполняемое действие или применяемая мера требуют определенных ресурсов: затрат финансов и времени. В крупных организациях такие ресурсы всегда ограничены и использовать их стоит с умом, а значит равномерно распределять их между различными областями и не допускать их перерасхода.

$$E = \sum_{t=0}^{T} |u_t|, E \leq E_{max}. \quad (14)$$

3) Далее следует расчет интенсивностей прироста количества уязвимостей и количества устраненных уязвимостей за шаг времени по формулам 15 и 16. Данные параметры неизменны в процессе эксплуатации защищенной системы, однако зависят от политик информационной безопасности в организации.

$$\lambda_t \approx \Delta k_t^+, \lambda_t \in [0, n - k_t] \quad (15)$$

$$\mu_t \approx \Delta k_t^-, \mu_t \in [0, k_t] \quad (16)$$

4) С учетом полученных ранее данных, по формуле 17, рассчитывается новое значение параметра k, которое будет использоваться на следующем шаге. Здесь также затрагиваются коэффициенты чувствительности к интенсивностям α и β, а

также здесь кроется первоначальный аспект управления риском: применение управляющего воздействия для уменьшения прироста количества критичных уязвимостей и увеличения интенсивности их устранения.

$$k_{t+1} = \lfloor \lambda_t \cdot (1 - \alpha u_t) \rfloor - \lfloor \mu_t \cdot (1 + \beta u_t) \rfloor \quad (17)$$

5) После проведения всех вычислений проводится расчет риска с учетом нормированного ущерба x и локального параметра чувствительности сегмента α_i по формуле 18. Включение этих параметров в формулу позволяет оценить влияние каждого из сегментов на суммарный риск.

$$Risk = x \cdot \alpha_i \frac{C_k^x C_{n-k}^{m-x}}{C_n^m} \quad (18)$$

6) После проведения вышеуказанных вычислений для всех L сегментов на каждом из шагов t, по формуле 19 рассчитывается суммарный риск. Здесь уже учитывается ценность каждого ресурса C_j при реализации атаки, что позволяет её перераспределять для непосредственного управления риском – удержания его в заданном диапазоне.

$$Risk_{sum} = \sum_{j=1}^{m} \left(C_j \sum_{i=1}^{L} Risk_{ij} \right) \quad (19)$$

7) Последним этапом является нормировка значений риска каждого из сегментов по формуле 20. Она проводится для корректной интерпретации результата, как вероятностного распределения.

$$Risk_{sum}^{norm} = \frac{1}{L} \sum_{i=1}^{L} Risk_i \quad (20)$$

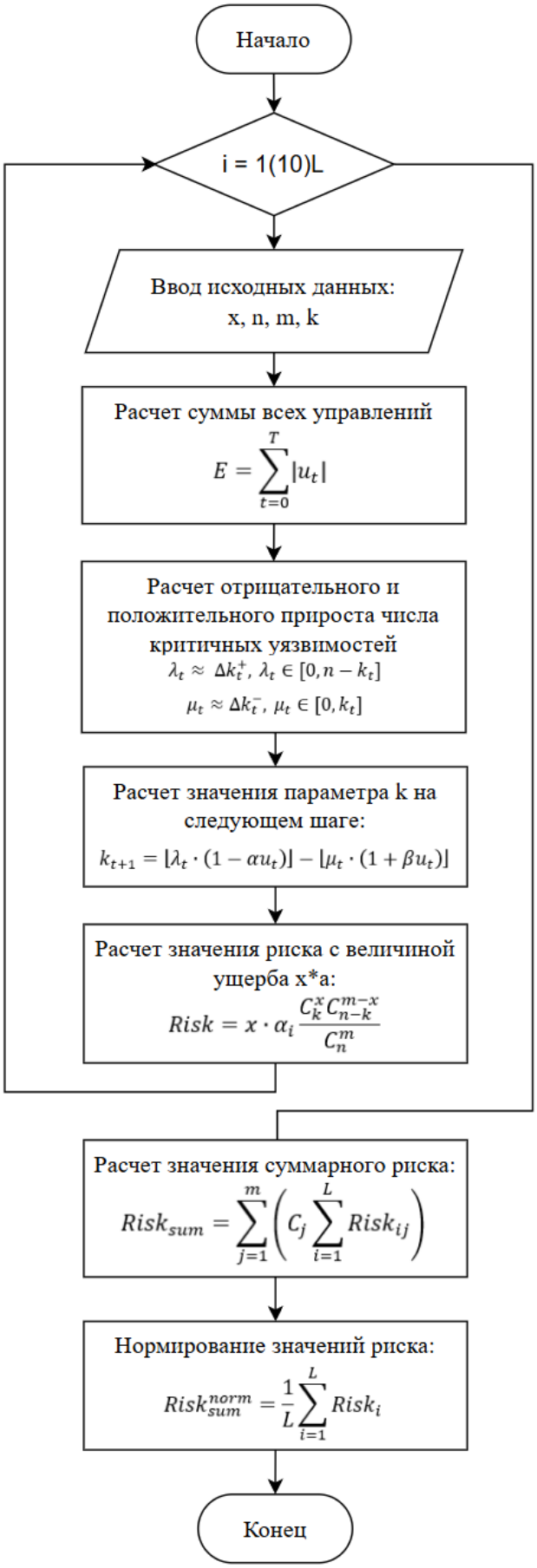


Рисунок 12 – Общий вид алгоритма управления рисками

2.2.4 Область применения алгоритма

Разработанный алгоритм найдет применение в сетях любого масштаба и может быть полезен для кибератак различных профилей. Он не имеет ограничений по количеству сегментов и потолочных значений параметров. Технически, алгоритм ограничен лишь спецификой входных данных и аппаратной частью вычислительной платформы, так как программная его реализация требует вычислительных мощностей.

Да, как и многие другие уже существующие алгоритмы, разработанный требует статистических данных. Однако его преимущество в том, что влияние этих данных не фундаментально ввиду наличия множества параметров, от которых зависит итоговая величина риска.

В умелых руках данный алгоритм может стать мощным инструментом для специалиста по информационной безопасности.

3 Проектирование программного обеспечения управления рисками

3.1 Общее описание архитектуры ПО

Программный комплекс реализован с глубокой проработкой лингвистических аспектов, обеспечивающих однозначность интерпретации параметров и результатов. Архитектурное решение основано на комбинации паттерна Model-View-Controller (MVC) и модульного подхода, который обеспечивает высокую совместимость и возможность расширения функционала. Программная реализация алгоритма состоит из следующих архитектурных слоёв:

1) Уровень представления (GUI):
   - Tkinter для оформления базовых элементов управления;
   - Matplotlib для профессиональной визуализации;
2) Уровень вычислений:

Вычислительное ядро (гипергеометрическая модель);

   - валидация входных параметров;
   - управление данными сегментов;
3) Уровень данных:
   - сбор и сохранение параметров сегментов;
   - результаты вероятностных расчетов.

В программном обеспечении применена инкапсуляция функционала, то есть логика работы системы сосредоточена в единственном классе, что значительно упрощает контроль над взаимодействием модулей друг с другом.

Зонирование интерфейса позволяет визуально четко разделить его на 3 независимые области (ввод данных, управление алгоритмом, визуализация). Графики формируются динамически в ответ на действия пользователя.

Проверка как формата данных, так и их семантической корректности обеспечивает двухуровневую валидацию значений.

При проектировании были соблюдены такие принципы, как: принцип единой ответственности, инверсия зависимостей, принцип открытости/закрытости.

3.2 Технологическое обеспечение

Программа построена с использованием современных технологий Python-экосистемы. Библиотечный ландшафт включает в себя такие библиотеки как:

– tkinter и ttk, отвечающие за создание кроссплатформенного GUI с нативным внешним видом;

– matplotlib – доступная визуализация сложных структур данных;

– numpy, упрощающая векторные вычисления и работу с числовыми массивами;

– math.comb, реализующая точные комбинаторные расчеты.

Программное обеспечения обладает низкими системными требованиями, что способствует общедоступности ПО и низкому порогу вхождения наряду с просто интеграцией. Запрос к системе следующий:

– Операционная система: Windows 10 или новее, Ubuntu 18.04 или новее, macOS Catalina или новее;

– Интерпретатор: Python 3.10 или новее (оптимизированный под математические вычисления)

– Аппаратные требования: процессор с поддержкой AVX, 4 ГБ ОЗУ, видеоадаптер и монитор, способные выводить изображение а разрешении 1920x1980.

Среди особенностей реализации ПО можно выделить потокобезопасность проводимых вычислений, адаптивную визуализацию для различных разрешений экранов и кэширование результатов расчетов.

ЗАКЛЮЧЕНИЕ

В рамках дипломного проектирования удалось достигнуть поставленной цели: повышение защищенности информационных сетей и систем за счет разработки алгоритмического и программного обеспечения управления рисками успешности реализации кибератак. В ходе выполнения поставленных задач были достигнуты следующие результаты:

1) Были рассмотрены существующие инструменты для управления рисками, определены их назначение, принципы и слабые стороны, что позволило сформировать требования к новому алгоритму

2) Разработаны два новых алгоритма управления рисками успешности реализации кибератак на основе биноминального и гипергеометрического распределения вероятностей. Новые алгоритмы максимально приближены к реальным условиям функционирования информационных систем.

3) Реализовано программное обеспечение для алгоритма на основе гипергеометрического распределения. Оно дружелюбно к пользователю, имеет минимальные системные требования, не требует интеграции и опыта использования.

Разработанные алгоритмическое и программное обеспечение стремится снизить нагрузку на специалистов в области информационной безопасности путём сокращения количества ручных вычислений, что поможет стабилизировать ситуацию на рынке труда в данной сфере, где явно выражена нехватка кадров.

Также результат работы может использоваться в качестве учебных материалов или основы для создания более масштабного, продуманного алгоритма, учитывающего максимально большое количество факторов.

СПИСОК ЛИТЕРАТУРЫ

1. Эпидемии в телекоммуникационных сетях / Остапенко [и др.]; [под ред. чл. – корр. РАН Д.А. Новикова]. – М: Горячая линия – Телеком, 2020. – 282 с. (Серия «Теория сетевых войн»; вып. 1). – Текст: непосредственный.

2. Атакуемые взвешенные сети / Остапенко [и др.]; [под ред. чл. – корр. РАН Д.А. Новикова]. – М: Горячая линия – Телеком, 2020. – 248 с. (Серия «Теория сетевых войн»; вып. 2). – Текст: непосредственный.

3. Социальные сети и деструктивный контент / Остапенко [и др.]; [под ред. чл. – корр. РАН Д.А. Новикова]. – М: Горячая линия – Телеком, 2020. – 274 с. (Серия «Теория сетевых войн»; вып. 3). – Текст: непосредственный.

4. Социальные сети и риск-мониторинг / Остапенко [и др.]; [под ред. чл. – корр. РАН Д.А. Новикова]. – М: Горячая линия – Телеком, 2020. – 266 с. (Серия «Теория сетевых войн»; вып. 4). – Текст: непосредственный.

5. Социальные сети и психологическая безопасность / Остапенко [и др.]; [под ред. чл. – корр. РАН Д.А. Новикова]. – М: Горячая линия – Телеком, 2021. – 232 с. (Серия «Теория сетевых войн»; вып. 5). – Текст: непосредственный.

6. Сетео-информационная эпидемиология / Остапенко [и др.]; [под ред. чл. – корр. РАН Д.А. Новикова]. – М: Горячая линия – Телеком, 2021. – 216 с. (Серия «Теория сетевых войн»; вып. 6). – Текст: непосредственный.

7. Картография защищаемого киберпространства / Остапенко [и др.]; [под ред. чл. – корр. РАН Д.А. Новикова]. – М: Горячая линия – Телеком, 2022. – 372 с. (Серия «Теория сетевых войн»; вып. 7). – Текст: непосредственный.

8. Организационно-правовая защита сетей / Остапенко [и др.]; [под ред. чл. – корр. РАН Д.А. Новикова]. – М: Горячая линия – Телеком, 2023. – 228 с. (Серия «Теория сетевых войн»; вып. 8). – Текст: непосредственный.

9. Автоматизированный атлас кибератак / Остапенко [и др.]; [под ред. чл. – корр. РАН Д.А. Новикова]. – М: Горячая линия – Телеком, 2025. – 188 с. (Серия «Теория сетевых войн»; вып. 9). – Текст: непосредственный.

10. Jack Freund. Measuring and Managing Information Risk: A FAIR Approach / Jack Freund, Jack Jones - Elsevier Science & Technology, 2014. – 408 с. – Текст: непосредственный.

11. Information security, cybersecurity and privacy protection — Guidance on managing information security risks. Общие технические условия: ISO/IEC 27005:2022: утвержден и введен: 14.09.2022 – Текст: непосредственный.

12. encyclopedia by Kaspersky. SOAR (Security Orchestration, Automation and Response) – URL: https://encyclopedia.kaspersky.ru/glossary/security-orchestration-automation-and-response-soar/ - Текст: электронный.

13. Косоруков О.А. Методы количественного анализа – Инфра-М: Институт экономики и финансов «Синергия», 2005. – 368 с. – Текст: непосредственный

14. Чернова Н. И. Теория вероятностей – Новосибирский Государственный Универститет, 2006 – 139 с. – Текст: непосредственный

15. Kaspersky Daily. Нехватка кадров в ИБ: корни проблемы и пути решения. 26.09.2024 – URL: https://www.kaspersky.ru/blog/cybersecurity-talent-shortage/38295/ - Текст: электронный

16. ISC2. 2024 ISC2 Cybersecurity Workforce Study. 31.10.2024 – URL: https://media.isc2.org/-/media/Project/ISC2/Main/Media/documents/research/ISC2_Cybersecurity_Workforce_Study_2023.pdf – Текст: электронный

17. Касперович С.А. Формализованные и комплексные методы прогнозирования и планирования. Брестский государственный технический университет, 2014 – 16 с. – Текст: непосредственный

18. Остапенко Г.А. Совершенствование организационно-правового обеспечения информационной безопасности предприятия: формирование риск-ландшафта сетевых атак / Г.А. Остапенко, Д.В. Щербакова, Т.Ю. Мирошниченко, А.А. Остапенко, А.Ю. Пекло // Информация и безопасность. 2023. - Т. 26. Вып. 2. - С. 203-210. – Текст: непосредственный.

19. Остапенко Г.А. Организационно-правовая защита от сетевых атак: методики формирования частных политик, регламентов и инструкций обеспечения безопасности организации (часть I) / Г.А. Остапенко, Д.В. Щербакова, Т.Ю.

Мирошниченко, А.А. Остапенко, А.С. Кривошеин // Информация и безопасность. 2023. - Т. 26. Вып. 3. - С. 329-340. – Текст: непосредственный.

20. Остапенко Г.А. Организационно-правовая защита от сетевых атак: методики формирования частных политик, регламентов и инструкций обеспечения безопасности организации (часть II) / Г.А. Остапенко, Д.В. Щербакова, Т.Ю. Мирошниченко, А.А. Остапенко, А.Г. Краснобородкин // Информация и безопасность. 2023. - Т. 26. Вып. 3. - С. 329-340. – Текст: непосредственный.

21. Остапенко Г.А. Организационно-правовая защита от сетевых атак: методики формирования частных политик, регламентов и инструкций обеспечения безопасности организации (часть III) / Г.А. Остапенко, Д.В. Щербакова, Т.Ю. Мирошниченко, А.А. Остапенко, А.Г. Краснобородкин // Информация и безопасность. 2023. - Т. 26. Вып. 3. - С. 341-358. – Текст: непосредственный.

22. Остапенко А.Г. Проектная деятельность: научно-методическое развитие в направлении внедрения средств искусственного интеллекта для обеспечения организационно-правовой защиты корпоративных сетей / А.Г. Остапенко, Д.В. Щербакова, А.А. Остапенко, Д.А. Нархов // Информация и безопасность. 2023. - Т. 26. Вып. 3. - С. 447-454. – Текст: непосредственный.

23. Остапенко Г.А. Нейросетевые задачи и компетенции проектной деятельности по созданию защищённых автоматизированных информационных систем / Г.А. Остапенко, А.П. Васильченко // Информация и безопасность. 2023. - Т. 26. Вып. 4. - С. 579-586. – Текст: непосредственный.

24. Остапенко Г.А. Методики регламентации обеспечения информационной безопасности атакуемых автоматизированных систем / Г.А. Остапенко, А.П. Васильченко // Информация и безопасность.2023. - Т.26.Вып. 4.- С.597-602. – Текст: непосредственный.

25. Остапенко А.Г. Направление совершенствования нормативно-правового обеспечения объектов информатизации на основе риск-анализа их кибербезопасности/ А.Г. Остапенко, Д.В. Щербакова, Д.А. Нархов, Ю.В. Макаров, А.С. Кривошеин // Информация и безопасность.2022. - Т.25.Вып. 4. - С.593-598. – Текст: непосредственный.

26. Остапенко Г.А. Формализация знаний и данных кибератак и уязвимостей / Г.А. Остапенко, А.П. Васильченко, А.А. Остапенко, Д.С. Покудин [и другие] // Информация и безопасность. 2024. - Т. 27. Вып. 2. - С. 231-238. – Текст: непосредственный.

27. Остапенко Г.А. Модернизация методического обеспечения автоматизированного сервиса агрегации данных и риск-анализа уязвимостей/ Г.А. Остапенко, А.П. Васильченко, А.А. Остапенко, Н.Н. Корвяков [и другие] // Информация и безопасность. 2024. - Т. 27. Вып. 2. - С. 219-230. – Текст: непосредственный.

28. Остапенко Г.А. Автоматизированный банк знаний и калькулятор рисков реализации кибератак и уязвимостей (часть I) / Г.А. Остапенко, А.П. Васильченко, А.А. Остапенко, Д.С. Нестеров, А.С. Дубов, В.А. Старцев // Информация и безопасность. 2024. Т. 27. Вып. 1. С. 7-30. – Текст: непосредственный.

29. Остапенко Г.А. Автоматизированный банк знаний и калькулятор рисков реализации кибератак и уязвимостей (часть II) / Г.А. Остапенко, А.П. Васильченко, А.А. Остапенко, Д.С. Нестеров, А.С. Дубов, В.А. Старцев // Информация и безопасность. 2024. Т. 27. Вып. 1. С. 31-54. – Текст: непосредственный.

30. Остапенко Г.А. Нейросетевой сервис регламентации мер противодействия кибератакам (часть I) / Г.А. Остапенко, А.П. Васильченко, А.А. Остапенко, А.А. Ноздрюхин, А.Г. Остапенко, И.Л. Батаронов, А.С. Кривошеин // Информация и безопасность. 2024. Т. 27. Вып. 3. С. 319-336. – Текст: непосредственный.

31. Остапенко Г.А. Нейросетевой сервис регламентации мер противодействия кибератакам (часть II) / Г.А. Остапенко, А.П. Васильченко, А.А. Остапенко, А.А. Ноздрюхин, А.Г. Остапенко, И.Л. Батаронов, Д.С. Печкин // Информация и безопасность. 2024. Т. 27. Вып. 3. С. 337-378. – Текст: непосредственный.

32. Остапенко Г.А. Использование средств искусственного интеллекта в ходе исследовательского и учебного процессов при подготовке специалистов по защите информации / Г.А. Остапенко, А.О. Калашников, А.Г. Остапенко, Е.А. Москалева,

Д.О. Карпеев, О.В. Поздышева, Ю.В. Макаров // Информация и безопасность. 2024. Т. 27. Вып. 3. С. 389-400. – Текст: непосредственный.

33. Методический документ от 28.10.2022г. ФСТЭК России. Методика оценки уровня критичности уязвимостей программных, программно-аппаратных средств. — Электрон. дан. — Режим доступа: https:// fstec.ru/dokumenty/vsedokumenty/spetsialnye-normativnye-dokumenty/ metodicheskij-dokument-ot-28-oktyabrya -2022-g-2. – Текст: электронный.

34. Сигорский В. П. Математический аппарат инженера. – К: Техника, 1977 - 768с. – Текст: непосредственный.

35. Асташов В.Н. Численные методы решения задач математической физики / В.Н. Асташов. — М.: Наука, 2003. – Текст: непосредственный.

36. Остапенко А.А. Методики и алгоритмы риск-анализа успешности реализации массированных кибератак / А.А. Остапенко // Информация и безопасность. 2024. Т. 27. Вып. 3. С. 401-420. – Текст: непосредственный.

37. 61 Остапенко А.Г. Картографирование киберпространства и обеспечение информационной безопасности / А.Г. Остапенко, А.Л. Сердечный, С.Д. Трубицын, Д.А. Нархов, В.Ю. Остапенко // Информация и безопасность. 2022. Т. 25. Вып. 1. С. 115-128. – Текст: непосредственный.

38. Остапенко Г.А. Матричная формализация целеполагания и результативности проектной деятельности в сфере защиты информации и обеспечения информационной безопасности / Г.А. Остапенко, В.И. Белоножкин, Н.И. Баранников, А.А. Остапенко, А.Г. Краснобородкин, Т.Ю. Мирошниченко // Информация и безопасность. 2022. Т. 25. Вып. 4. С. 475-484. – Текст: непосредственный.

39. Сердечный А.Л. Риск-анализ и прогнозирование частоты и ущербности компьютерных атак / А.Л. Сердечный, А.С. Маликова, А.Г. Остапенко, М.Е. Волкова, Д.А. Нархов, А.Н. Бартенев. // Информация и безопасность. 2021. Т. 24. Вып. 2. С. 159-178. – Текст: непосредственный.

40. Остапенко А.Г. Методические основы проектной деятельности при выполнении научно-исследовательской работы студентами специалитета в сфере

обеспечения информационной безопасности / А.Г. Остапенко, А.С. Пахомова, Д.А. Нархов, А.А. Остапенко, А.И. Шеншин // Информация и безопасность. 2023. Т. 26. № 2. С. 169-176. – Текст: непосредственный.

41. Кобцев О.Е. Сетевые атаки компьютерными вирусами: риск-ландшафт / О.Е. Кобцев, А.И. Шеншин, В.М. Питолин, Л.В. Паринова, В.Г. Юрасов, Д.С. Печкин // Информация и безопасность. 2024. Т. 27. Вып. 1. С. 55-66. – Текст: непосредственный.

42. Рытов М.Ю. Порядок оценки уровня эффективности системы непрерывного противодействия инцидентам информационной безопасности на объекте / М.Ю. Рытов, О.М. Голембиовская, Е.В. Кондрашова, М.М. Голембиовский, К.Е. Шинаков // Информация и безопасность. 2024. Т. 27. Вып. 1. С. 135-142. – Текст: непосредственный.

43. Александров А.Г. Анализ угроз информационной безопасности при использовании облачных сервисов / А.Г. Александров, А.Ю. Петухов, М.Ю. Рытов // Информация и безопасность. 2024. Т. 27. Вып. 1. С. 143-151. – Текст: непосредственный.

44. Щербакова Д.В. Организационно-правовое обеспечение сетевой контрразведывательной деятельности корпорации (часть I) / Д.В. Щербакова, А.Ю. Пекло, А.С. Кривошеин, В.М. Питолин, О.Ю. Макаров, В.Н. Кострова // Информация и безопасность. 2024. Т. 27. Вып. 2. С. 185-196. – Текст: непосредственный.

45. Громов Ю.Ю. Изучение возможности виртуализации технических средств защиты объектов с применением имитационного моделирования / Ю.Ю. Громов, П.И. Карасев, Ф.М. Пыршев // Информация и безопасность. 2024. Т. 27. Вып. 3. С. 379-388. – Текст: непосредственный.

46. Подтопельный В.В. Исследование специфики моделирования компьютерных атак с использованием марковских процессов принятий решений и Qобучения / В.В. Подтопельный // Информация и безопасность. 2024. Т. 27. Вып. 3. С. 421-440. – Текст: непосредственный.

47. Зимницкий А.Г. Программно-технический комплекс выявления субъектов с повышенным риском девиантного поведения / А.Г. Зимницкий, А.А

Остапенко // Информация и безопасность. 2022. Т. 25. № 4. С. 571-582. – Текст: непосредственный.

48. Остапенко А.Г. Целеполагание проектной деятельности по созданию инструментария автоматизированного выявления и риск-анализа деструктивных контентов, аффилированных с персоналом корпораций / А.Г. Остапенко, И.А, Боков, С.В. Лихобабин, Д.С. Ясенко, Е.Ю. Чапурин // Информация и безопасность. 2023. Т. 26. № 4. С. 465-494. – Текст: непосредственный.

ПРИЛОЖЕНИЕ А

```
from math import comb
import matplotlib.pyplot as plt
import numpy as np
from tkinter import *
from tkinter import ttk, messagebox
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg


class HypergeometricApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Вычисление рисков")
        self.root.geometry("1000x700")


        self.segments = []
        self.create_widgets()


    def create_widgets(self):
        # Основные фреймы
        self.input_frame = ttk.LabelFrame(self.root, text="Ввод параметров сегмента",
padding=10)
        self.input_frame.pack(side=LEFT, fill=Y, padx=5, pady=5)
```

```
        self.segments_frame = ttk.LabelFrame(self.root, text="Добавленные сегменты", padding=10)
        self.segments_frame.pack(side=LEFT, fill=Y, padx=5, pady=5)

        self.plot_frame = ttk.LabelFrame(self.root, text="Графики распределения", padding=10)
        self.plot_frame.pack(side=RIGHT, fill=BOTH, expand=True, padx=5, pady=5)

        # Поля ввода для параметров сегмента
        ttk.Label(self.input_frame, text="Общее число уязвимостей (n):").grid(row=0, column=0, sticky=W, pady=2)
        self.n_entry = ttk.Entry(self.input_frame)
        self.n_entry.grid(row=0, column=1, pady=2)

        ttk.Label(self.input_frame, text="Критичные уязвимости (k):").grid(row=1, column=0, sticky=W, pady=2)
        self.k_entry = ttk.Entry(self.input_frame)
        self.k_entry.grid(row=1, column=1, pady=2)

        ttk.Label(self.input_frame, text="Число атак (m):").grid(row=2, column=0, sticky=W, pady=2)
        self.m_entry = ttk.Entry(self.input_frame)
        self.m_entry.grid(row=2, column=1, pady=2)
```

```
        # Кнопки управления
        ttk.Button(self.input_frame, text="Добавить сегмент",
command=self.add_segment).grid(row=3, column=0, columnspan=2, pady=10)
        ttk.Button(self.input_frame, text="Построить графики",
command=self.plot_distributions).grid(row=4, column=0, columnspan=2, pady=5)
        ttk.Button(self.input_frame, text="Очистить все",
command=self.clear_all).grid(row=5, column=0, columnspan=2, pady=5)

        # Таблица добавленных сегментов
        self.segments_tree = ttk.Treeview(self.segments_frame, columns=("n", "k", "m"),
show="headings", height=15)
        self.segments_tree.heading("n", text="n")
        self.segments_tree.heading("k", text="k")
        self.segments_tree.heading("m", text="m")
        self.segments_tree.column("n", width=80, anchor=CENTER)
        self.segments_tree.column("k", width=80, anchor=CENTER)
        self.segments_tree.column("m", width=80, anchor=CENTER)
        self.segments_tree.pack()

        # Кнопка удаления выбранного сегмента
        ttk.Button(self.segments_frame, text="Удалить выбранное",
command=self.remove_segment).pack(pady=5)
```

```
        # Область для графиков
        self.figure = plt.Figure(figsize=(8, 5), dpi=100)
        self.canvas = FigureCanvasTkAgg(self.figure, master=self.plot_frame)
        self.canvas.get_tk_widget().pack(fill=BOTH, expand=True)

    def hypergeometric_distribution(self, n, k, m, x):
        return comb(k, x) * comb(n - k, m - x) / comb(n, m)

    def add_segment(self):
        try:
            n = int(self.n_entry.get())
            k = int(self.k_entry.get())
            m = int(self.m_entry.get())

            if k < 10:
                messagebox.showerror("Ошибка", f"x (10) не может быть больше k ({k})")
                return
            if (m - 10) > (n - k):
                messagebox.showerror("Ошибка", f"m - x ({m} - 10) не может быть больше n
- k ({n - k})")
                return

            self.segments.append((n, k, m))
```

```
            self.segments_tree.insert("", END, values=(n, k, m))

            # Очищаем поля ввода
            self.n_entry.delete(0, END)
            self.k_entry.delete(0, END)
            self.m_entry.delete(0, END)

        except ValueError:
            messagebox.showerror("Ошибка", "Пожалуйста, введите целые числа для всех параметров")

    def remove_segment(self):
        selected_item = self.segments_tree.selection()
        if selected_item:
            index = self.segments_tree.index(selected_item[0])
            self.segments.pop(index)
            self.segments_tree.delete(selected_item)

    def clear_all(self):
        self.segments = []
        for item in self.segments_tree.get_children():
            self.segments_tree.delete(item)
        self.figure.clear()
```

```
        self.canvas.draw()


    def plot_distributions(self):
        if not self.segments:
            messagebox.showwarning("Предупреждение", "Нет данных для построения графиков")
            return

        self.figure.clear()
        ax = self.figure.add_subplot(111)

        x_values = np.arange(0, 11)
        total_probs = np.zeros(11)
        colors = plt.cm.tab10.colors

        for i, (n, k, m) in enumerate(self.segments, 1):
            y_values = []
            for x in x_values:
                try:
                    prob = self.hypergeometric_distribution(n, k, m, x)
                    y_values.append(prob)
                except ValueError:
                    y_values.append(0)
```

```
    total_probs += np.array(y_values)

    ax.plot(x_values, y_values,
            marker='o',
            linestyle='-',
            linewidth=1,
            markersize=4,
            color=colors[i % len(colors)],
            alpha=0.7,
            label=f'Сегмент {i}: n={n}, k={k}, m={m}')

avg_probs = total_probs
ax.plot(x_values, avg_probs,
        marker='s',
        linestyle='--',
        linewidth=2,
        markersize=8,
        color='black',
        label=f'Среднее по {len(self.segments)} сегментам')

ax.set_title('Гипергеометрическое распределение по сегментам')
```

```
        ax.set_xlabel('Количество критических уязвимостей (x)')
        ax.set_ylabel('Вероятность P(X=x)')
        ax.set_xticks(x_values)
        ax.grid(True, linestyle='--', alpha=0.5)
        ax.legend(bbox_to_anchor=(1.05, 1), loc='upper left')

        self.figure.tight_layout()
        self.canvas.draw()

if __name__ == "__main__":
    root = Tk()
    app = HypergeometricApp(root)
    root.mainloop()
```